

Sampling NNLO QCD phase space with normalizing flows

Timo Janßen,^{a,b} Rene Poncelet,^c Steffen Schumann^b

^a*Campus-Institut Data Science, Georg-August-Universität Göttingen, Goldschmidtstraße 1, 37077 Göttingen, Germany*

^b*Institut für Theoretische Physik, Georg-August-Universität Göttingen, Friedrich-Hund-Platz 1, 37077 Göttingen, Germany*

^c*Institute of Nuclear Physics, ul. Radzikowskiego 152, 31-342 Krakow, Poland*

E-mail: timo.janssen@theorie.physik.uni-goettingen.de,
rene.poncelet@ifj.edu.pl,
steffen.schumann@phys.uni-goettingen.de

ABSTRACT: We showcase the application of neural importance sampling for the evaluation of NNLO QCD scattering cross sections. We consider Normalizing Flows in the form of discrete Coupling Layers and time continuous flows for the integration of the various cross-section contributions when using the sector-improved residue subtraction scheme. We thereby consider the stratification of the integrands into their positive and negative contributions, and separately optimize the phase-space sampler. We exemplify the novel methods for the case of gluonic top-quark pair production at the LHC at NNLO QCD accuracy. We find significant gains with respect to the current default methods used in STRIPPER in terms of reduced cross-section variances and increased unweighting efficiencies. In turn, the computational costs for evaluations of the integrand needed to achieve a certain statistical uncertainty for the cross section can be reduced by a factor 8.

KEYWORDS: Phase-space sampling, Monte Carlo integration, NNLO QCD calculations

Contents

1	Introduction	1
2	Phase-space sampling for scattering processes	4
2.1	Phase-space integration basics	4
2.2	Integrands in the sector-improved residue subtraction scheme	7
3	Normalizing Flow improved sampling for NNLO QCD	11
3.1	Coupling Layer flows	12
3.2	Continuous Normalizing Flows	12
3.3	Training of Normalizing Flows	14
3.4	Conditional Flows	16
4	Application to top-quark pair production at NNLO QCD	17
4.1	Computational setup	17
4.2	Integrators and optimization	17
4.3	Results	20
5	Conclusions	28
A	Numerical results	31

1 Introduction

Fixed-order perturbative calculations provide unique means to address scattering processes at high energy scales as probed in particle colliders. In particular when considering hadronic initial states, higher-order QCD predictions are vital for the success of experiments aiming to scrutinize our understanding of fundamental-particle interactions. Such fixed-order calculations require the evaluation of squared transition matrix elements in dependence on the initial- and final-state particle momenta. Besides the construction of compact expressions for the matrix elements, the evaluation of physical cross sections requires the efficient generation of phase-space configurations for the external states. The computation of tree-level cross sections has been fully automated in modern event-generator programs even for high-multiplicity processes [1–5]. While such leading-order (LO) squared matrix elements are strictly positive, this is no longer guaranteed at higher orders, e.g. due to interference contributions of Born and loop diagrams. Next-to-leading order (NLO) calculations can also be considered fully automated for Standard Model processes, thanks to

dedicated loop-amplitude generators such as OPENLOOPS [6, 7], RECOLA [8, 9] or MADLOOP [10]. Another vital ingredient for numerical evaluations of NLO cross sections are infrared subtraction methods such as Frixione–Kunszt–Signer (FKS) [11] and Catani–Seymour (CS) subtraction [12, 13]. These accomplish the separate cancellation of soft- and/or collinear singularities in the real-emission and virtual corrections, thereby rendering them finite, opening the possibility to integrate both parts numerically. Efficient phase-space sampling for NLO calculations demands suitable treatment of real-emission configurations, probing both, hard and well separated emissions, as well as rather soft and collinear kinematics.

Moving to next-to-next-to-leading order (NNLO) introduces further challenges. On the one hand, the computation of the required two-loop matrix elements is more complicated since no numerical method has been developed yet, necessitating (semi-)analytic derivations on a case-by-case basis. For example, state-of-the-art computations derive amplitudes with two-to-three kinematics [14–21]. The second bottleneck arises from the organization of the cancellation of the infrared divergences in terms of subtraction or slicing methods. At present, several approaches are being explored. Each presents unique advantages and challenges and differs in generality and efficiency. Slicing methods like q_T [22, 23] and N-jettiness [24–31] slicing are easy to implement but typically rather inefficient. The slicing parameter also might restrict the type of processes the scheme can be applied for, like q_T -slicing, which works only for colour-singlet or massive final-state particles but not for jet production. N-jettiness slicing gets increasingly challenging if more than three coloured particles are involved due to the complex colour structures. NNLO subtraction methods follow the paradigm of the successful FKS and CS NLO schemes and implement local subtraction terms. Roughly speaking, the various approaches differ in three fundamental properties: identification of the singular integrable limits and construction of suitable subtraction terms, phase-space mappings, and the treatment of integrated subtraction terms (numerical or analytical). One can broadly separate FKS-like (the sector-improved residue [32–35], nested soft-collinear [36–40], local analytic [41–43] subtraction schemes), CS-like (antenna [44–62], CoLoRfulNNLO [63–67]) and projection-to-Born schemes [68, 69].

For the feasibility of NNLO QCD calculations, the numerical efficiency of the employed subtraction scheme is a critical aspect. The typical computational costs are $\mathcal{O}(10^3)$ CPUh for simple colourless final states. If the final state is more complicated, particularly if it involves massless coloured particles, the resource demand increases. For example, an NNLO QCD computation for the hadronic production of vector-boson plus jet final states is already of $\mathcal{O}(10^4)$ CPUh, a di-jet computation typically required $\mathcal{O}(10^5)$ and three-jet calculations $\mathcal{O}(10^6)$ CPUh. These estimates apply for typical LHC fiducial phase spaces and single-differential observables. If higher-dimensional distributions or particular corners of phase space are considered, the costs might be even higher. The details depend on the scheme used in the compu-

tation, but usually, the main bottleneck turns out to be the phase-space integration of the double real-emission contribution and the numerical cancellation between various (integrated) subtraction terms and between different contributions, which require a very small residual statistical uncertainty on the individual components.

Accordingly, it is desirable to increase the efficiency of such numerical computations. Two avenues can be explored to achieve this: 1) Modifying the subtraction scheme in order to minimize the cancellations between contributions or 2) improving the numerical-integration techniques. In this work, we explore the second approach motivated by the fact that many NNLO QCD calculational frameworks use relatively simple techniques for numerical integration. To better adapt to the given integrand, it is common to use techniques like integration-variable re-mappings (importance sampling) and multi-channel phase-space parameterizations. However, the standard method to perform phase-space integration is to fall back to rather simple adaptive Monte Carlo techniques such as VEGAS [70].

The rapid advances of modern machine-learning techniques have sparked renewed interest and research activities in phase-space integration and sampling methods for high-energy physics event generators [71]. In particular, this includes the development of Neural Importance Sampling methods [72, 73] that employ trainable Normalizing-Flow transformations [74–77] to optimize the density of phase-space integration variables. Such methods have been developed in the context of the SHERPA event generator [78, 79], the MADNIS approach [80–82] for MADGRAPH5_AMC@NLO [83, 84], see also Refs. [85–89]. In essence, these methods offer potential to reduce the variance of cross-section integral estimates by reducing the spread of event weights. This will typically also yield improved unweighting efficiencies for the generation of unit-weight events as they are customarily used in experimental applications of LO and NLO accurate theory predictions. Recently, there have been first attempts to produce (partially) unweighted events also for NNLO QCD predictions [90]. For novel approaches to accelerate event unweighting, i.e. rejection sampling, through the use of neural-network matrix-element surrogates, see Refs. [91, 92].

In this publication, we pioneer the application of Neural Importance Sampling for phase-space integration in a complete NNLO QCD computation¹. As an initial example, we consider the fully differential hadronic production of top-quark pairs in the STRIPPER framework [95], based on sector improved residue subtraction. To this end, we train and apply dedicated Normalizing-Flow samplers for the various components of the NNLO calculation, including the Born, single and double real-emission corrections, as well as one- and two-loop contributions. We here consider splitting the target distribution in positive and negative contributions and separately optimize

¹Applications of Normalizing Flows for solving Feynman-parameter integrals, as they appear in higher-order perturbative calculations, have been presented in [93, 94].

samplers for the two pieces, resulting in a significant cross-section variance reduction and accordingly the computing resources needed to arrive at an accuracy target. For the Normalizing-Flow transformations we consider Coupling-Layer architectures [96] as well as Continuous Normalizing Flows [97]. For the latter, we build on the experience and implementation from Ref. [98], where orders of magnitude improvements in unweighting efficiency were achieved for the LO case using Flow Matching [99, 100] in conjunction with the portable event generator PEPPER [101]. The corresponding results get compared to what is obtained with variable re-mappings based on VEGAS, what closely resembles the default approach used in STRIPPER [102].

The article is structured as follows: In Sec. 2 we introduce the basic concepts of phase-space integration and sampling, including the generation of unweighted samples. We furthermore introduce the STRIPPER approach to perform NNLO QCD calculations, thereby focusing on the structure of the contributing phase-space integrals. In Sec. 3 we briefly introduce Coupling-Layer and Continuous Normalizing Flows, describe our training approach, and how discrete integration variables, e.g. corresponding to helicity states of external particles, can be incorporated in conditional flows. Ultimately, in Sec. 4 we present the setups of our numerical calculations and the results obtained for gluonic top-quark pair production at NNLO QCD. Our conclusions and a brief outlook are compiled in Sec. 5.

2 Phase-space sampling for scattering processes

In this section we briefly recapitulate basic elements of Monte Carlo integration and importance sampling specifically. In particular we consider integrands of non-definite sign and study the impact of their stratification into positive and negative contributions. Further, in Sec. 2.2 we introduce the STRIPPER approach to NNLO QCD calculations and thereby put the focus on the appearing phase-space integrals and their parameterization.

2.1 Phase-space integration basics

We consider a generic integration problem over some d -dimensional hypercube Ω of unit volume with a piece-wise Riemann integrable integrand $f : \Omega \rightarrow \mathbb{R}, \mathbf{x} \mapsto f(\mathbf{x})$:

$$I = \int_{\mathbf{x} \in \Omega} d\mathbf{x} f(\mathbf{x}). \quad (2.1)$$

The Monte Carlo estimate for the integral and the associated one-sigma statistical error for N independent uniformly distributed points $\mathbf{x}_i \in \Omega$ read

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N f(\mathbf{x}_i), \quad \delta \hat{I} = \sqrt{\frac{1}{N-1} \left(\frac{1}{N} \sum_{i=1}^N f^2(\mathbf{x}_i) - \hat{I}^2 \right)}. \quad (2.2)$$

To reduce the variance of the integral estimate, the density of sampling points can be adjusted to better resemble the target distribution $f(\mathbf{x})$, known as *importance sampling* [103]. To this end, a bijective mapping $\mathbf{H} : \Omega \rightarrow \Omega, \mathbf{x} \mapsto \mathbf{H}(\mathbf{x})$ can be considered, resulting in

$$I = \int_{\mathbf{H}(\mathbf{x}) \in \Omega} d\mathbf{H} \frac{f(\mathbf{x})}{h(\mathbf{x})}, \quad (2.3)$$

with the density $h : \Omega \rightarrow \mathbb{R}, \mathbf{x} \mapsto h(\mathbf{x})$ given by the Jacobian determinant

$$h(\mathbf{x}) = \left| \det \left(\frac{\partial \mathbf{H}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|. \quad (2.4)$$

The density function $h(\mathbf{x})$ might thereby depend on a set of parameters that can be adjusted in an initial optimization phase to optimally map the target distribution $f(\mathbf{x})$. However, in order to be able to efficiently generate phase-space points distributed according to $h(\mathbf{x})$, typically the *inverse transform method* [103, 104] is used. This, however, requires an easily integrable form for $h(\mathbf{x})$, with an invertible cumulative distribution, what might limit the expressivity and thus the performance of the algorithm.

Should the integrand not be strictly positive, during this initial phase typically its modulus is considered for optimizing the sampling distribution $h(\mathbf{x})$. However, in the actual integration phase the sign of the target is kept and propagated accordingly. In this situation there exists a lower limit for the variance that can be derived from the case that $|f(x)|/h(x) = w = \text{const.}$ In this extreme case, the mean is given by

$$\hat{I} = w \frac{N_+ - N_-}{N} \equiv w(2\alpha - 1), \quad (2.5)$$

where $\alpha = N_+/N$, with $N_{\pm}$ denoting the number of points ending up in the positive or negative region of phase space, respectively. For the variance we find

$$\text{Var}(\hat{I}) = w^2 - w^2(2\alpha - 1)^2 = w^2(4\alpha(1 - \alpha)). \quad (2.6)$$

The relative uncertainty on the integral is then given by

$$\frac{\delta \hat{I}}{\hat{I}} = \frac{1}{\sqrt{N-1}} \frac{\sqrt{\alpha(1-\alpha)}}{\alpha - \frac{1}{2}}. \quad (2.7)$$

For $\alpha < 1$ this yields a non-vanishing lower bound for the relative error, independent of how well $h(x)$ approximates $f(x)$. The limit on the error $\delta \hat{I}$ we denote by δ_{opt} .

Stratification for signed integrands

A possible alternative treatment, that we will explore in this paper, is to actually split the integrand into its positive and negative contributions, i.e.

$$f(\mathbf{x}) = f_+(\mathbf{x}) + f_-(\mathbf{x}), \quad \text{with} \quad f_{\pm}(\mathbf{x}) = \Theta(\pm f(\mathbf{x}))f(\mathbf{x}), \quad (2.8)$$

and to separately adapt suitable densities $h_{\pm}(\mathbf{x})$, with $h_{\pm} : \Omega \rightarrow \mathbb{R}, \mathbf{x} \mapsto h_{\pm}(\mathbf{x})$. We use Θ to denote the Heaviside step function. Accordingly, the desired integral is given by

$$I = \int_{\mathbf{H}_+(\mathbf{x}) \in \Omega} d\mathbf{H}_+ \frac{f_+(\mathbf{x})}{h_+(\mathbf{x})} + \int_{\mathbf{H}_-(\mathbf{x}) \in \Omega} d\mathbf{H}_- \frac{f_-(\mathbf{x})}{h_-(\mathbf{x})}. \quad (2.9)$$

This can be regarded as an instance of *stratified sampling* [103] with two strata. In this case, the Monte Carlo estimate of the integral becomes

$$\hat{I}_{\text{strat}} = \hat{I}_+ + \hat{I}_- = \frac{1}{N_+} \sum_{i=1}^{N_+} \frac{f_+(\mathbf{x}_i)}{h_+(\mathbf{x}_i)} + \frac{1}{N_-} \sum_{i=1}^{N_-} \frac{f_-(\mathbf{x}_i)}{h_-(\mathbf{x}_i)}. \quad (2.10)$$

In contrast to above, there is no lower limit on the variance. If $h_{\pm}$ approximate their respective targets $f_{\pm}$ perfectly, the integral can be determined with vanishing error, i.e. $\delta_{\text{opt}} = 0$. In the (realistic) case of finite variance, we need to consider the sum of the two variances weighted by their respective relative fractions, resulting in

$$\delta \hat{I}_{\text{strat}} = \sqrt{\frac{1}{N-1} \left[\frac{N}{N_+} \text{Var}(\hat{I}_+) + \frac{N}{N_-} \text{Var}(\hat{I}_-) \right]} \quad \text{with} \quad (2.11)$$

$$\text{Var}(\hat{I}_{\pm}) = \frac{1}{N_{\pm}} \sum_{i=1}^{N_{\pm}} \left(\frac{f_{\pm}(\mathbf{x}_i)}{h_{\pm}(\mathbf{x}_i)} \right)^2 - \hat{I}_{\pm}^2, \quad (2.12)$$

where we used $N = N_+ + N_-$. This error can be smaller than the one using a single $h(\mathbf{x})$ optimized on $|f(\mathbf{x})|$, provided that the individual variances $\text{Var}(\hat{I}_{\pm})$ are small enough. This requires not only that the densities $h_{\pm}(\mathbf{x})$ adapt well to the functional shape of $f_{\pm}(\mathbf{x})$ but also that they efficiently sample within the volumes $\Omega_{\pm} \subset \Omega$ where the target $f_{\pm}(\mathbf{x})$ is non-zero. In terms of the latter property, Normalizing Flows should have an advantage over VEGAS, as they are not limited to a factorized approach.

Unweighted event generation

In order to generate unit-weight events from the target function, a *rejection-sampling algorithm* [105] is employed. An event with weight $w_i = f(\mathbf{x}_i)$, i.e. $w_i = f(\mathbf{x}_i)/h(\mathbf{x}_i)$ when sampling with a non-uniform density, gets accepted with probability

$$p_i = \frac{w_i}{w_{\max}}, \quad (2.13)$$

with $w_{\max}$ a pre-determined maximal weight. Accordingly, the efficiency of the unweighting procedure is determined by

$$\epsilon_{\text{uw}} \equiv \frac{\langle w \rangle}{w_{\max}}, \quad (2.14)$$

with $\langle w \rangle$ the average event weight in a given sample. The inverse $1/\epsilon_{\text{uw}}$ thereby determines the average number of target-function evaluations needed to generate a single unit-weight event. Note that for not strictly positive target functions the unit weights carry a sign. For discussions on the sources of negative event weights in high-energy event simulations at NLO QCD accuracy, their impact on the statistical properties of event samples, and ideas how to mitigate their contribution, see for example Refs. [106–109].

To soften the dependence of the unweighting efficiency on rare outlier events featuring very large weights, the naively obtained maximum can systematically be reduced, e.g. by removing the largest weights up to a certain percentile of the total integral [73, 91]. Correspondingly, events whose weight w_i exceeds the reduced maximum will pass the unweighting step and be assigned an overweight given by

$$w_i^{\text{overweight}} = \text{sgn}(w_i) \left| \frac{w_i}{w_{\text{max}}^{\text{reduced}}} \right|. \quad (2.15)$$

2.2 Integrands in the sector-improved residue subtraction scheme

In this section we briefly review the anatomy of NNLO QCD calculations based on the sector-improved residue subtraction scheme as implemented in the STRIPPER framework. We thereby focus on the structure of the appearing phase-space integrals to compute (differential) cross sections. Scattering cross sections are obtained upon integration of transition matrix elements over the momenta of external particles. The crucial point is that eventually, employing a suitable parameterization, the appearing integrals can be related to integrals over a unit hypercube in the sense of Eq. (2.1).

The construction starts from the expression of the (differential) hadronic cross section of producing a final state X in collinear factorization, given by

$$\sigma(h(P_1)h(P_2) \rightarrow X) = \sum_{ab} \int \int_0^1 dx_1 dx_2 \phi_a(x_1) \phi_b(x_2) \hat{\sigma}_{ab}(x_1 P_1, x_2 P_2), \quad (2.16)$$

where we have suppressed the dependence on the factorization and renormalization scales. Here the sum of a (b) runs over all massless parton flavours, i.e. gluons g and quarks $q \in \{u, d, c, s, b\}$, as well as their anti-particles $\bar{q}$. The functions ϕ_a denote the corresponding renormalized Parton-Distribution-Functions (PDFs). $\hat{\sigma}_{ab}$ represents the partonic cross section, whose expansion in the strong coupling constant α_s up to NNLO can be written as

$$\hat{\sigma}_{ab} = \hat{\sigma}_{ab}^{(0)} + \hat{\sigma}_{ab}^{(1)} + \hat{\sigma}_{ab}^{(2)} + \mathcal{O}(\alpha_s^{n_{(0)}} \alpha_s^3). \quad (2.17)$$

Powers of the strong coupling (including the corresponding tree-level coupling order $n_{(0)}$) are thereby absorbed in the expansion coefficients. The leading-order or Born-level contribution reads

$$\hat{\sigma}_{ab}^{(0)} = \hat{\sigma}_{ab}^{\text{B}}. \quad (2.18)$$

At NLO QCD there appear virtual ($\hat{\sigma}_{ab}^V$), real emission ($\hat{\sigma}_{ab}^R$), and, for hadronic initial states, collinear PDF counterterm ($\hat{\sigma}_{ab}^C$) contributions. Thus, the complete NLO correction reads

$$\hat{\sigma}_{ab}^{(1)} = \hat{\sigma}_{ab}^R + \hat{\sigma}_{ab}^V + \hat{\sigma}_{ab}^C. \quad (2.19)$$

At NNLO QCD double-virtual ($\hat{\sigma}_{ab}^{VV}$), double-real ($\hat{\sigma}_{ab}^{RR}$), and real-virtual ($\hat{\sigma}_{ab}^{RV}$) contributions have to be taken into account, potentially together with collinear PDF counterterms ($\hat{\sigma}_{ab}^{C1}, \hat{\sigma}_{ab}^{C2}$). Accordingly, the NNLO QCD correction can be written as:

$$\hat{\sigma}_{ab}^{(2)} = \hat{\sigma}_{ab}^{RR} + \hat{\sigma}_{ab}^{RV} + \hat{\sigma}_{ab}^{VV} + \hat{\sigma}_{ab}^{C1} + \hat{\sigma}_{ab}^{C2}. \quad (2.20)$$

For each contribution i we denote σ^i , i.e. with the PDF convolution in Eq. (2.16) performed, as the hadronic contribution to the cross section.

Besides the various contributions from real and virtual corrections, the partonic cross section is furthermore composed of different partonic final states that need to be summed over. We absorb all these contributions in a generic index $j \in \mathcal{C}_{ab}^i(ab \rightarrow X)$, where $\mathcal{C}_{ab}^i(ab \rightarrow X)$ is the set of indices required for a given contribution i and Born process $ab \rightarrow X$, accordingly

$$\hat{\sigma}_{ab}^i = \sum_{j \in \mathcal{C}_{ab}^i(ab \rightarrow X)} \hat{\sigma}_{ab,j}^i. \quad (2.21)$$

The individual contributions can be cast in the form

$$\hat{\sigma}_{ab,j}^i = N_j \int d\Phi_j \mathcal{F}_{ab,j}(\Phi_j), \quad (2.22)$$

with Φ_j the Lorentz-invariant phase space², and $\mathcal{F}_{ab,j}$ a function of the kinematics, representing transition matrix elements and subtraction terms, as well as phase-space cuts.

Consider, as an example, the LO case: the function $\mathcal{F}_{ab,j}$ is simply a matrix element times a measurement function (F), used to define infrared-safe observables and the fiducial phase space:

$$\mathcal{F}_{ab,j} = |\mathcal{M}_{ab,j}(\Phi_j)|^2 F(\Phi_j). \quad (2.23)$$

At higher orders, real-radiation contributions require the subtraction of infrared limits and their combinations with virtual infrared divergences. In the STRIPPER scheme, we introduce for this purpose selector functions, see [34]. In the case of a single-real emission, where j represents an $n + 1$ partonic process, we have:

$$\hat{\sigma}_{ab}^{(1)} \ni \int d\Phi_j \sum_{kl} \mathcal{S}_{kl} |\mathcal{M}_j(\Phi_j)|^2 F(\Phi_j). \quad (2.24)$$

²This phase-space measure might include the integration over convolution variables in case of collinear factorization counterterms.

Selector functions have the property that they sum up to unity:

$$\sum_{kl} \mathcal{S}_{kl} = 1, \quad (2.25)$$

and that they vanish sufficiently fast in infrared limits not given by the soft limit of parton k and the collinear limit $k \rightarrow l$. This property allows one to explicitly parameterize these limits in terms of variables ξ (energy of k) and η (angle between k and l). Details on the phase-space parameterization can be found in Ref. [35]. After the transformation and parameterization of the rest of the phase space, we find integrals of the form (now for each sector individually):

$$\hat{\sigma}_{ab}^{(1)} \ni \int \int_0^1 \frac{d\xi}{\xi^{1-2\varepsilon}} \frac{d\eta}{\eta^{1-\varepsilon}} \underbrace{\int d\tilde{\Phi}_j \mathcal{S}_{kl} |\mathcal{M}_j(\Phi_j)|^2 F(\Phi_j)}_{\text{finite} \equiv f_{kl}^j(\xi, \eta)}. \quad (2.26)$$

This form makes it explicit that the only remaining singularities are given by the poles in ξ and η . The plus distribution can be employed to extract these poles:

$$\begin{aligned} \hat{\sigma}_{ab}^{(1)} \ni & \int \int_0^1 d\xi d\eta \frac{f_{kl}^j(\xi, \eta) - f_{kl}^j(0, \eta) - f_{kl}^j(\xi, 0) + f_{kl}^j(0, 0)}{\xi^{1-2\varepsilon} \eta^{1-\varepsilon}} \\ & + \frac{1}{2\varepsilon} \int_0^1 d\eta \frac{f_{kl}^j(0, \eta) - f_{kl}^j(0, 0)}{\eta^{1-\varepsilon}} \\ & + \frac{1}{\varepsilon} \int_0^1 d\xi \frac{f_{kl}^j(\xi, 0) - f_{kl}^j(0, 0)}{\xi^{1-2\varepsilon}} \\ & + \frac{1}{2\varepsilon^2} f_{kl}^j(0, 0), \end{aligned} \quad (2.27)$$

what yields a set of finite integrals that can be evaluated by numerical integration after expansion in ε . Integrals with poles cancel against virtual corrections, and only their ε^0 coefficient contributes to the cross section. The construction of the (integrated) subtraction terms is expressed through splitting and soft-functions and lower-order matrix elements as detailed in [34, 35]. In the light of numerical integration, it is important to note that integrals with poles (i.e. integrated subtraction terms) are much easier to evaluate and integrate than the actual subtracted finite term (the first line in Eq. (2.27)), since they only consist of lower-multiplicity matrix elements and any remaining subtraction is in variables that do not change the kinematics if the parameterization is suitably chosen [35].

A similar construction can be performed at NNLO QCD for the double-real-emission contribution [34, 35]. Focusing on the more challenging finite part, taking the limit $\varepsilon \rightarrow 0$ (which can be done as all divergencies are properly subtracted) and including also the parameterization of the finite integrals in terms of a hypercube, the integration problem can be reformulated as

$$\hat{\sigma}_{ab}^{(i)} \ni \int_{[0,1]^m} d^m \chi \int_{[0,1]^n} d^n \mathbf{x} \frac{f_{\{k\}}^j(\chi, \mathbf{x}) - \sum f_{\{k\}}^j(\chi|_{\rightarrow 0}, \mathbf{x})}{\prod_i^m \chi_i} \equiv \int_{[0,1]^{n+m}} dy g_{\{k\}}^j. \quad (2.28)$$

Here, the selector indices are shown in a compressed notation $\{k\}$ (each $\{k\}$ corresponds to a set of indices, i, j for single-collinear, i, j, k for triple-collinear and i, j, k, l for double-collinear sectors). The total number of singular phase-space integrals m is 2 for one emission and 4 for two emissions, respectively. The collection of subtraction terms is denoted by $\sum f_{\{k\}}^j(\chi|_{\rightarrow 0}, \mathbf{x})$, where the sum runs over all possibilities to distribute zeros in the χ vector. Formulated this way and including the PDF convolutions as well as the summation over all sectors and partonic channels, it is natural to view the final integration problem in a form which resembles Eq. (2.1):

$$\sigma^i = \sum_{ab} \sum_{j \in \mathcal{C}_{ab}^i(ab \rightarrow X)} \sum_{\{k\}} \int_{[0,1]^{n+2}} dx_1 dx_2 d\mathbf{y} g_{\{k\}}^j(\mathbf{y}). \quad (2.29)$$

This procedure, as discussed in more detail in Ref. [35], splits the contributions appearing in Eqs. (2.19) and (2.20) further into pieces that can individually be numerically integrated. They are sorted into the following groups:

$$\sigma^R = \sigma^{RF} + \sigma^{RU}, \quad (2.30)$$

$$\sigma^V = \sigma^{VF} + \sigma^{VU}, \quad (2.31)$$

$$\sigma^{RR} = \sigma^{RRF} + \sigma^{RRSU} + \sigma^{RRDU}, \quad (2.32)$$

$$\sigma^{RV} = \sigma^{RVF} + \sigma^{RVDU} + \sigma^{RVFR}, \quad (2.33)$$

$$\sigma^{VV} = \sigma^{VVF} + \sigma^{VVDU} + \sigma^{VVFR}. \quad (2.34)$$

To increase the computational efficiency, a couple of optimizations are performed:

- **Sector sampling** is used to evaluate the sum over sectors, i.e. all choices for $\{k\}$. This is realized by randomly choosing the sector for a Monte Carlo point (based on some fixed and optimized weights) or as an additional one-dimensional integration.
- **Helicity sampling** is employed for real-emission corrections to reduce the computational time spent on matrix-element evaluations. The sectors identify a set of partons (e.g. i and j in the case of single-collinear sectors) for which the sum over helicities is performed explicitly:

$$\int_{[0,1]^n} d\mathbf{y} g_{\{k\}}^j(\mathbf{y}) = \sum_{h \in \text{hel}_{\{k\}}} \int_{[0,1]^n} d\mathbf{y} g_{\{k\},h}^j(\mathbf{y}). \quad (2.35)$$

This sum can be taken out and performed via Monte Carlo methods similar to the sector sampling.

- **Mapping of PDF channels** can be used to reduce the sum over the 121 parton-flavour combinations in Eq. (2.16) to a smaller set of generic processes.

When there are two (or more) pairs of initial-state partons (a, b) and (a', b') , where

$$\hat{\sigma}_{ab}(x_1 P_1, x_2 P_2) = \hat{\sigma}_{a'b'}(x_1 P_1, x_2 P_2), \quad (2.36)$$

the expression can be rewritten as

$$\sigma(h(P_1)h(P_2) \rightarrow X) = \sum_{c \in \mathcal{C}} \iint_0^1 dx_1 dx_2 \left(\sum_{(ab) \in c} \phi_a(x_1) \phi_b(x_2) \right) \hat{\sigma}_{ab}, \quad (2.37)$$

i.e. the sum over PDFs is performed for each phase-space point.

Ultimately, we sequence the discrete selection choices to a single discrete index l , which represents a specific combination of $a, b, j, \{k\}, h$ in Eq. (2.29) such that:

$$\sum_{ab} \sum_{j \in \mathcal{C}_{ab}^i(ab \rightarrow X)} \sum_{\{k\}, h} \int_{[0,1]^n} d\mathbf{y} g_{\{k\}, h}^j(\mathbf{y}) = \sum_l \int_{[0,1]^n} d\mathbf{y} g_l(\mathbf{y}). \quad (2.38)$$

The sum over l can again be performed in a Monte Carlo fashion.

In addition to the numerical integration of the (differential) cross section, the samples can be (partially) unweighted as discussed at the end of Sec. 2.1. For example, such an unweighting of NNLO QCD partonic events has been performed in the context of the HighTEA project [90]. A separate maximum weight $w_{\max}$ for the positive and negative regions of the phase space is used for the HighTEA procedure, and a reduction of the maximum weight according to Eq. (2.15) is tuned to achieve an unweighting efficiency of $\sim 0.1\%$ for all contributions. The negative and positive regions of phase space are unweighted with their own respective maximum weight. The admixture of positive and negative events and different contributions is chosen to optimize the numerical integration uncertainty for some benchmark differential distributions for a given process.

3 Normalizing Flow improved sampling for NNLO QCD

In this section we describe how the mapping $\mathbf{H}$, introduced in Sec. 2.1 for transporting a simple base density to a density that is closer to the target, can be realized by Normalizing Flows. Through specific architecture choices, explained below, Normalizing Flows can be designed in a way so that their Jacobian determinant is tractable without having to invert the neural networks. Therefore, neural networks of arbitrary complexity, trained by stochastic gradient descent, can be used. In Sec. 3.1, we describe flows based on Coupling Layers, which implement the flow as a chain of discrete steps. As an alternative approach, Sec. 3.2 deals with Continuous Normalizing Flows, where the flow is regarded as a transformation continuous in time. The training of both kinds of flows, specific to our application, is described in Sec. 3.3. Lastly, we consider flows conditioned on discrete inputs in Sec. 3.4.

3.1 Coupling Layer flows

In general, Normalizing Flows provide a bijective mapping between two probability densities. Typically, they map from a simple base density, e.g. a Gaussian or uniform one, to a complex density that approximates a target. The key idea of Normalising Flows can best be understood from the change of variables formula for probability densities:

$$\mathbf{y}_1 = \phi(\mathbf{y}_0) \quad \Rightarrow \quad \log q(\mathbf{y}_1) = \log q_0(\mathbf{y}_0) - \log \left| \det \frac{\partial \phi}{\partial \mathbf{y}_0} \right|. \quad (3.1)$$

A simple base density q_0 can be transformed into a complicated density q by constructing a suitable flow $\phi : \Omega \rightarrow \Omega$. The main difficulty is the computation of the Jacobian determinant $\det \frac{\partial \phi}{\partial \mathbf{y}_0}$. In general, this has a cubic cost in n , the number of dimensions of $\mathbf{y}$. However, this can be reduced by constraining the flow, for which different approaches are possible. Here we consider Coupling Layers [96], which split the input into two partitions A and B and transform them according to

$$\begin{aligned} \mathbf{y}_1^A &= \mathbf{y}_0^A, \\ \mathbf{y}_1^B &= C(\mathbf{y}_0^B; m_\theta(\mathbf{y}_0^A)), \end{aligned} \quad (3.2)$$

where m_θ is a neural network with trainable parameters θ and the coupling transform C is an invertible function that transforms $\mathbf{y}_0^B$ element-wise.

For spline-based flows [110], we restrict the input domain of C to $[0, 1]^{|B|}$, with $|B|$ the cardinality of partition B , and divide each dimension into K bins. In the form of a spline, piece-wise functions defined in each bin are joined into a continuous curve. Here we consider Neural Spline Flows [111], which use rational-quadratic splines, whose parameters (bin widths, bin heights, derivatives at internal knots) are determined by the neural-network output $m_\theta(\mathbf{y}_0^A)$.

As can be seen from Eq. (3.2), a single Coupling Layer transforms only a part of the input. To obtain an expressive transformation, several layers can be composed in a chain $\phi = \phi_k \circ \dots \circ \phi_1$ with permutations of the variables in-between. To capture all possible correlations, one needs at least $2\lceil \log_2 n \rceil$ coupling layers if $n > 5$ and n layers if $n \leq 5$ [85].

Coupling Layers based on spline transformations naturally live on the unit hypercube Ω . This means we can simply use a uniform base distribution with constant density q_0 on Ω and use the Coupling Layer flow to map it to a non-uniform distribution on Ω . According to Eq. (3.1), the model density is then given by the Jacobian determinant of the flow.

3.2 Continuous Normalizing Flows

An alternative to flows based on a chain of Coupling Layers is given by Continuous Normalizing Flows, which provide a flow $\phi_t : [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ continuous in a “time”

parameter $t \in [0, 1]$. The change of variable now reads

$$\log q_t(\mathbf{y}_t) = \log q_0(\mathbf{y}_0) - \log \left| \det \frac{\partial \phi_t}{\partial \mathbf{y}_0} \right|, \quad (3.3)$$

where q_t is a time-dependent probability density function. The flow itself is constructed implicitly via the ordinary differential equation (ODE)

$$\frac{d}{dt} \phi_t(\mathbf{y}) = v_t(\phi_t(\mathbf{y})), \quad \text{with} \quad \phi_0(\mathbf{y}) = \mathbf{y}, \quad (3.4)$$

where $v_t : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a time-dependent vector field. A neural network $v_{t;\theta}(\mathbf{y})$, with trainable parameters θ , can be used to model the vector field [97]. As the probability is locally conserved, this vector field needs to obey a continuity equation.

To generate a point $\mathbf{y}_1$ with the flow ϕ_t , we need to sample an initial point $\mathbf{y}_0 \sim q_0$ and integrate the ODE (3.4) over time:

$$\mathbf{y}_1 = \phi_1(\mathbf{y}_0) = \int_0^1 v_t(\phi_t(\mathbf{y}_0)) dt. \quad (3.5)$$

In practice, this can be done approximately with a numerical ODE solver. If, vice versa, we want to evaluate the model probability density at a given point $\mathbf{y}_1$, we need to solve the reversed ODE

$$\frac{d}{ds} \begin{bmatrix} \phi_{1-s}(\mathbf{y}) \\ f(1-s) \end{bmatrix} = \begin{bmatrix} -v_{1-s}(\phi_{1-s}(\mathbf{y})) \\ -\text{div}(v_{1-s}(\phi_{1-s}(\mathbf{y}))) \end{bmatrix} \quad (3.6)$$

for $s \in [0, 1]$, with the initial conditions given by

$$\begin{bmatrix} \phi_1(\mathbf{y}) \\ f(1) \end{bmatrix} = \begin{bmatrix} \mathbf{y}_1 \\ 0 \end{bmatrix}. \quad (3.7)$$

The solution for the model log probability is then found to be

$$\log q_1(\mathbf{y}_1) = \log q_0(\mathbf{y}_0) - f(0). \quad (3.8)$$

Eqs. (3.6)–(3.8) follow from the continuity equation [99].

In contrast to spline-based CL flows, ODE flows are in general defined on $\mathbb{R}^n$ instead of the unit hypercube Ω . To deal with this, we use the sigmoid function $\sigma_{\text{sigmoid}} : \mathbb{R} \rightarrow \Omega, x \mapsto \sigma_{\text{sigmoid}}(x) = 1/(1 + e^{-x})$ and its inverse, the logit function, to bijectively map between the two domains. For the ODE flow, the base density q_0 is a standard normal. To sample a point in Ω , we draw a point $\mathbf{y}_0 \in \mathbb{R}^n$ from q_0 , use the flow to map it to a point $\mathbf{y}_1 \in \mathbb{R}^n$ according to Eq. (3.5) and then map it to $\sigma_{\text{sigmoid}}(\mathbf{y}_1) \in \Omega$. The overall density on Ω is given by the base density $q_0(\mathbf{y}_0)$ multiplied by the Jacobian determinants of the two maps. For training with data living on Ω , they need to be mapped to $\mathbb{R}^n$ first with the logit transform. Note that in order to avoid numerical problems at the edges of phase space, we use a simple scale and shift transformation, $x \mapsto x \cdot (1 - \epsilon) + \epsilon/2$, where ϵ is a small number, so that the regions where the logit function is steepest are avoided. Throughout this work, we use $\epsilon = 1 \times 10^{-6}$.

3.3 Training of Normalizing Flows

Since we can evaluate the model probability of a Normalizing Flow – both for the discrete, Coupling Layer based, and the continuous flow – we can train it by minimizing the Kullback–Leibler (KL) divergence [112] between the target density, here denoted $p(\mathbf{y})$, and the model density $q_\theta(\mathbf{y})$, which in the long term is equivalent to maximum-likelihood estimation³ The KL divergence is asymmetric in its arguments and in practice we prefer the forward version,

$$D_{\text{KL}}(p \parallel q_\theta) = \int d\mathbf{y} p(\mathbf{y}) \log \left(\frac{p(\mathbf{y})}{q_\theta(\mathbf{y})} \right) = \mathbb{E}_{p(\mathbf{y})} \left[\log \left(\frac{p(\mathbf{y})}{q_\theta(\mathbf{y})} \right) \right], \quad (3.9)$$

due to its mode covering behaviour. This avoids severe local underestimation of the probability density, which would lead to huge importance weights w and therefore high variance of the integral estimate $\hat{I}$, cf. Eq. (2.2), and low unweighting efficiency ϵ_{uw} , cf. Eq. (2.14). However, Eq. (3.9) assumes that we sample $\mathbf{y} \sim p(\mathbf{y})$. Since our integration method is based on importance sampling, we do not actually produce data from the target density p . Instead, we sample $\mathbf{y} \sim q_\theta(\mathbf{y})$ with weights $w(\mathbf{y}) = p(\mathbf{y})/q_\theta(\mathbf{y})$, where the weights must necessarily be taken into account in order to reproduce the correct target distribution. Generating unweighted data through a rejection-sampling step would be too inefficient for the training. However, as shown in Ref. [110], we can find a suitable objective by looking at the gradient of D_{KL} with respect to the model parameters:

$$\begin{aligned} \nabla_\theta D_{\text{KL}}(p \parallel q_\theta) &= \nabla_\theta \int d\mathbf{y} p(\mathbf{y}) \log \left(\frac{p(\mathbf{y})}{q_\theta(\mathbf{y})} \right) \\ &= - \int d\mathbf{y} p(\mathbf{y}) \nabla_\theta \log q_\theta(\mathbf{y}) \\ &= - \int d\mathbf{y} q_\theta(\mathbf{y}) \frac{p(\mathbf{y})}{q_\theta(\mathbf{y})} \nabla_\theta \log q_\theta(\mathbf{y}) \\ &= \mathbb{E}_{q_\theta(\mathbf{y})} \left[- \frac{p(\mathbf{y})}{q_\theta(\mathbf{y})} \nabla_\theta \log q_\theta(\mathbf{y}) \right]. \end{aligned} \quad (3.10)$$

This means we can train on weighted data by simply weighting the negative log-likelihood with the importance weight $w(\mathbf{y}) = p(\mathbf{y})/q_\theta(\mathbf{y})$ while ignoring its gradient in the optimization. The objective requires a tractable target density $p(\mathbf{y})$ but does not require its gradient, which means the flow can be adapted to targets that cannot easily be made differentiable, like those in large existing code bases.

³The KL-divergence requires normalized densities as input. The fact that the normalization of the cross section is a priori unknown does not pose a problem in practice, only the derivative of the KL-divergence with respect to the model parameters is needed for optimization. For gradient-descent optimizers like Adam that dynamically set their step sizes the unknown normalization factor cancels out. See, for example, Ref. [110].

Due to the appearance of the importance weights in the objective, we can expect slow convergence as long as q is far away from p . For this reason, we employ an iterative training scheme in Sec. 4.2, where a flow trained on a limited set of data is used to produce the next set of training data with better distribution. This is similar to the approach proposed in Ref. [113]. However, there the authors use a diffusion model and add new data to a buffer in each iteration, while we use flow matching and replace the data in each iteration. There are also similarities to the buffered training used by MADNIS [80]. The main difference to our approach is that they perform the training in a single run with a buffer of weighted events that is continuously replaced. In contrast to that, we use a fixed set of training data during each iteration, which gets completely replaced for the next iteration. The advantage is that we can separate the production of the training data from the model training. This allows us to use the hardware efficiently, since STRIPPER is running on CPUs while we train and run our models on GPUs. In each iteration, data is pushed to the GPU memory only once, minimizing overhead for input/output operations. Furthermore, it allows us to design a minimalistic interface to STRIPPER, as it only needs to read in random numbers and Jacobians and write out weights. Disadvantages are the disk space required to store the event samples and the more complicated workflow due to the iterations. However, batch processing on a compute cluster simplifies the latter considerably.

For Continuous Normalizing Flows, the use of a numerical ODE solver makes training and evaluation of the density slow in comparison to Coupling Layer flows. This motivated the development of simulation-free training methods [99, 114, 115], which do not require an explicit simulation during training. The idea is to instead directly match the vector field $v_{t;\theta}(\mathbf{y})$ to a target vector field $u_t(\mathbf{y})$ which generates the target density $p_1(\mathbf{y})$ through a target probability density path $p_t(\mathbf{y})$. This is intractable in general since $u_t(\mathbf{y})$ is unknown. However, we can construct conditional probability paths and vector fields which are conditioned on individual samples from the target distribution and the base distribution. This leads to the Conditional Flow Matching objective [100]

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,q(\mathbf{z}),p_t(\mathbf{y}|\mathbf{z})} \|v_{t;\theta}(\mathbf{y}) - u_t(\mathbf{y}|\mathbf{z})\|^2, \quad (3.11)$$

where $\mathbf{z}$ is a latent conditioning variable. Here, we use the independent coupling, where $q(\mathbf{z}) = q(\mathbf{y}_0)q(\mathbf{y}_1)$ with $\mathbf{y}_0$ and $\mathbf{y}_1$ being samples from the base and the target distribution, respectively. The conditionals are Gaussian flows between $\mathbf{y}_0$ and $\mathbf{y}_1$ with standard deviation σ , given by

$$p_t(\mathbf{y}|\mathbf{z}) = \mathcal{N}(\mathbf{y} | t\mathbf{y}_1 + (1-t)\mathbf{y}_0, \sigma^2), \quad (3.12)$$

$$u_t(\mathbf{y}|\mathbf{z}) = \mathbf{y}_1 - \mathbf{y}_0. \quad (3.13)$$

The standard deviation σ serves as a hyperparameter that introduces noise into the training. Remarkably, minimizing Eq. (3.11) is equivalent to a regression of

$v_t(\mathbf{y}; \theta)$ to the (unknown) marginal target vector field $u_t(\mathbf{y})$, since it has the same gradients [99, 100].

Similar to above, we have to account for the fact that in our application, $\mathbf{y}_1$ is sampled from the flow instead of the target. We can easily correct for this by introducing the importance weight as a multiplicative factor into Eq. (3.11). This is a variant of the Energy Conditional Flow Matching objective proposed in Ref. [100]. We then use the same iterative training scheme as introduced above for the Coupling Layer flow.

3.4 Conditional Flows

As described in Sec. 2.2, phase-space sampling in STRIPPER involves sampling over a discrete index l , representing a combination of sector and helicity. This can be implemented by a categorical distribution, for which the probabilities of the categories can be adapted to data based on their contribution to the total cross section of the integral with index l . At the same time, the amplitudes to be evaluated are conditioned on l . Traditionally, the integrator (and its VEGAS mapping) is blind to this and only adapts to the marginal amplitudes. While it would be possible to adapt individual VEGAS mappings for each index l , a poor convergence would be expected at least for those indices where few events are generated due to their low probabilities.

In contrast to VEGAS, Normalizing Flows can themselves be conditioned on l and are able to learn the correlations between different indices. This is particularly interesting in view of the fact that the amplitude distributions for different indices are often quite similar to each other. Sometimes, they only differ in the marginal distribution of one parameter. Therefore, learning the conditional amplitudes can be expected to be more efficient with respect to the amount of training data than learning each marginal amplitude individually.

Technically, the conditional Normalizing Flow provides a conditional model density $q_\theta(\mathbf{y}|l)$. For the Coupling Layer Flow, this means that each coupling transform C in Eq. (3.2) has to be conditioned on l . This can be achieved by conditioning the neural networks, such that $m_\theta = m_\theta(\mathbf{y}_0^A|l)$. In consequence, the conditioning variable l is independently fed into each coupling layer. For Continuous Normalizing Flows, only the vector field is conditioned, such that $v_{t;\theta} = v_{t;\theta}(\mathbf{y}|l)$.

Since the index l is categorical, it needs to be transformed to a continuous representation before being processed by a neural network. We use embedding layers for this purpose. There are different possibilities to feed the representation into the network. For simplicity, we concatenate the representation of l and the other input variables and jointly feed them into the input layer.

4 Application to top-quark pair production at NNLO QCD

To apply, validate and benchmark our novel integration methods for NNLO QCD calculations in the STRIPPER approach, we consider hadronic top-quark pair production. To ease the calculations, we here focus on the purely gluonic channel, i.e. no contributions from light-quark flavours are included, what dramatically reduces the number of partonic channels. This simplification allows us to keep the presentation rather compact, while already including a large set of infrared limits appearing at NNLO QCD. We compare the results obtained for the various cross-section contributions, and in particular the achieved variances, for different phase-space sampling techniques, including the default approach used in STRIPPER, and in particular the Normalizing-Flow methods introduced in Sec. 3. We thereby consider training separate samplers for the positive and negative contributions to the integrands, requiring accurate predictions of the sign of the target function. As a second figure of merit, we consider the corresponding unweighting efficiencies that determine the resources needed to obtain unit-weight events.

4.1 Calculational setup

We consider proton–proton collisions at $\sqrt{s} = 13$ TeV, corresponding to the conditions of Run II at the LHC. For the proton PDFs the NNPDF31_NNLO_AS_0118 set [116] is used. We consider the final-state top quarks to be on-shell using $m_t = 173.3$ GeV. The mass parameter is thereby renormalized on-shell. The renormalization (μ_R) and factorization (μ_F) scales we set to the same fixed value

$$\mu_R = \mu_F = m_t/2. \quad (4.1)$$

The renormalization of the strong coupling is performed in the 5-flavour scheme, assuming as an input value $\alpha_s(M_Z^2) = 0.118$, consistent with the PDF set.

For illustration, in Fig. 1 we present example Feynman diagrams contributing to gluonic top-quark pair production at LO $\propto \mathcal{O}(\alpha_s^2)$, NLO $\propto \mathcal{O}(\alpha_s^3)$, and NNLO $\propto \mathcal{O}(\alpha_s^4)$, thereby indicating to which STRIPPER contribution they belong to, cf. Eqs. (2.30)–(2.34). Note, for the here considered scale setting, i.e. $\mu_R = \mu_F$, the contributions σ^C , σ^{C1SU} , and σ^{C2DU} vanish. Furthermore, σ^{VU} , σ^{VVDU} , and σ^{VVFR} generally contribute to ϵ -poles only. Accordingly, we omit these contributions in the following discussion. Also, these contributions are negligible in terms of their computational complexity and thus do not significantly affect our computational efficiency considerations.

4.2 Integrators and optimization

In Sec. 2.2, we already discussed the structure of the integrands and the phase-space parameterization in the STRIPPER formalism and how they can be cast into

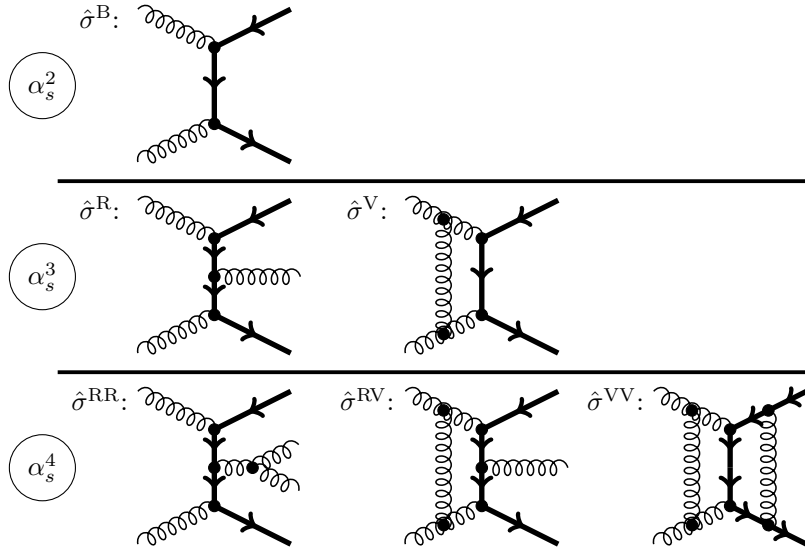


Figure 1. Representative Feynman diagrams contributing to gluonic top-quark pair production at NNLO QCD.

integrals over unit-hypercubes, see Eq. (2.38). Apart from the discrete variable l , these integrals are precisely in the form of Eq. (2.1).

In practice, we developed a Python interface to the STRIPPER framework, and the sampling and training procedures are steered through a Python script. From the interface point of view, a random point from the unit-hypercube with a weight (either uniform or transformed through the integrator) needs to be provided. This point is passed to the STRIPPER C++ code, which returns the full event weight. In the case of the flow-based models, we treat the index l as a conditional parameter to the neural networks we train, as explained in Sec. 3.4. During sampling, we use precomputed weights, based on the contribution to the total cross section of the integral with index l , to sample the index l . In the case of the VEGAS integrator, this is achieved by treating l as an additional discretized dimension.

Apart from providing the unit-hypercube random samples, we want to stress that the integrand is handled entirely within the STRIPPER framework. This allows access to all established facilities to compute differential cross sections, scale variations and unweighting for the HIGHTEA [90] database.

We study the variance-reduction capability of three different adaptive phase-space samplers when applied to gluonic $t\bar{t}$ production in the STRIPPER framework:

- **VEGAS:** The VEGAS importance-sampling algorithm [70] as implemented in the Python package [117] is used. 100 channels, i.e. discrete bins, are used per phase-space dimension. This integrator has been cross-checked against a one-dimensional version of the PARNI algorithm [102], which presents the default for phase-space integration within the STRIPPER framework for NNLO QCD

calculations. Since the performance of these two integrators was evaluated to be practically identical, we only show results for VEGAS in this work.

- **CL:** A Normalizing Flow sampler based on Coupling Layers as described in Sec. 3.1 is used. We employ the NFlows package [118] to implement piecewise rational-quadratic Coupling Layers [111] with 16 spline knots per dimension. The number of Coupling Layers is given by $2\lceil\log_2 n\rceil$, where n is the number of phase-space dimensions. Between two Coupling Layers, the inputs are permuted with a binary permutation scheme. Each coupling transformation is parameterized by a multilayer perceptron (MLP) with a structure that depends on the number of phase-space dimensions, which determines the number of input and output variables. The number and size of hidden layers is also variable. All hidden layers have twice as many nodes as the preceding layer, so that their size is increasing exponentially. The largest hidden layer has more nodes than the output layer, but less than twice as many. In our examples, this results in MLPs with up to 3 layers and flows with $\sim 100\text{k}$ to $\sim 20\text{M}$ trainable parameters. We use ReLU activation functions for the hidden layers. The discrete parameter l is incorporated as a conditional parameter, as described in Sec. 3.4. It is transformed into a continuous representation by an embedding layer, where the number of nodes is equal to the number of possible values of l . We concatenate this representation to the other input variables in each Coupling Layer.
- **ODE:** Corresponds to the Continuous Normalizing Flow sampling as described in Sec. 3.2. For training the time-dependent vector field, we use the implementation of the Flow-Matching method in the TORCHCFM package [100, 119]. The vector field is parameterized by an MLP with four hidden layers of width 512, using SELU activation functions [120]. This results in models with roughly 1M trainable parameters. The time variable t is concatenated to the other input variables. The discrete index l is one-hot encoded and then also concatenated to the other inputs. During training, the time t is sampled from a power-law distribution $p_\alpha(t) \propto t^{1/(1+\alpha)}$, $t \in [0, 1]$, as proposed in [121]. We set $\alpha = 1$. For the parameter σ appearing in Eq. (3.12), we use a value of $\sigma = 1 \times 10^{-4}$. For using the vector field in a Continuous Normalizing Flow and to sample from the flow, we use the TORCHDYN package [122]. The sampling requires to numerically solve an ODE, for which we use the Dormand–Prince method [123] of order 5 and set the absolute and relative tolerances to 1×10^{-4} .

The quoted settings (e.g. number of Coupling Layers, number of neural-network parameters, or channels) result from some simple and not too-extensive hyperparameter optimization.

Both types of flow are implemented and trained with the use of the PYTORCH library [124]. We use the ADAMW optimizer [125] with a learning rate of 1×10^{-3} . For the CL flow, we reduce the learning rate on plateaus. To facilitate an efficient training of the flow-based integrators on GPUs, we perform the training iteratively. For the first training iteration, we generate a uniformly distributed sample $\mathbf{y}$ including the weight and discrete selection variable l , and write it to disk.

The training procedure depends on the flow model. For the CL flow, the data is split into a training (80%) and validation set (20%), which we found necessary to avoid over-fitting. We train for up to 600 epochs while we allow it to stop if a plateau in the validation loss is detected. Each epoch is split into mini-batches based on randomly permuted subsets of the training data with a size of about 10k events. In the case of the ODE flow, we train on the complete data set for 600 epochs with a mini-batch size of about 100k. The large mini-batch size reduces the variance of the estimator for the flow-matching loss, Eq. (3.11), leading to faster convergence. Finally, the trained model is used to generate a new set of phase-space points $\mathbf{y}$ and prior weights. The integrand is evaluated at these points, and the weights are written to disk, which enables the next training iteration. At each iteration, we load the previous iteration of the model as the starting point and determine new weights for the categorical distribution of the index l .

How much a given model (with fixed hyperparameters) can reduce the variance depends on two main aspects: 1) how much training data is used and 2) the expressivity of the parameterization. The first aspect is rather evident, but the second is more subtle. Assuming an infinite amount of training data, it is clear that a given model might not be able to describe the integrand arbitrarily well. This is because the parameterization might not be able to capture the actual functional form or correlations between variables (most evident for VEGAS).

We consider the same amount of training data to compare different models, i.e. the same number of phase-space points, which we choose to be 10^7 . However, the training protocols depend on the model. The VEGAS methods are updated every 10^4 points while the flow-based models are trained on 10^6 points in large batches. After the model is trained, all parameters are frozen, and the integrand is evaluated with 10^6 points. These samples are then used to investigate the variance and convergence.

4.3 Results

We performed the above training procedures on all integrals contributing to gluonic top-quark pair production at NNLO QCD, both, on the absolute and the positive/negative split integrands. After the training, i.e. adaptation, of the three sampling methods, we performed the numerical integration with 10^6 points and based our analyses on the corresponding event samples. To validate the integrators and benchmark their performance, we computed the following quantities:

- $\hat{\sigma}$ and $\delta\hat{\sigma}$: the Monte Carlo estimate for the full hadronic cross section and its statistical uncertainty based on the integrator trained on the absolute value of the integrand.
- $\hat{\sigma}^\pm$ and $\delta\hat{\sigma}^\pm$: the Monte Carlo estimate and uncertainty for the positive and negative contribution to the full integral, respectively, based on the integrator separately trained on the positive/negative integrand.
- $\hat{\Sigma}^\pm$ and $\delta\hat{\Sigma}^\pm$: The sum of $\hat{\sigma}^+$ and $\hat{\sigma}^-$, the uncertainty is thereby rescaled to reflect a sample size of 10^6 points in total. The ratio of effective points for the positive contribution, cf. Eq. (2.5), is given by:

$$\alpha = \frac{\sqrt{\text{Var}(\hat{\sigma}^+)}}{\sqrt{\text{Var}(\hat{\sigma}^+)} + \sqrt{\text{Var}(\hat{\sigma}^-)}}.$$

- $\epsilon_\Phi^\pm$: the efficiency to correctly hit the positively/negatively contributing phase-space region $\Phi^\pm$ in case of positive/negative split integrand.
- $\epsilon^\pm$: the unweighting efficiency for the positive and negative contribution, respectively.
- $\epsilon_{0.1\%}^\pm$: the unweighting efficiency after the reduction of the respective $w_{\max}$ based on removing the largest weights that contribute less than 0.1% of the corresponding integrated cross section, see Sec. 2.1 for details.

The results for the various contributions are collected in Tab. 1 in App. A for the Born and NLO terms and in Tabs. 2 and 3 for the double-real and virtual NNLO corrections, respectively. Alongside, we present an estimate for the CPU cost for a single point (relative to the evaluation of a σ^B point) and the estimated optimal uncertainty δ_{opt} when training on the absolute integrand. All integral estimates are statistically compatible within a few multiples of their respective statistical uncertainty, a critical validation criterion.

Integral-variance reduction: training on the absolute integrand

The prime performance characteristics of an integrator are its expressiveness and adaptability to the integrand, which should be reflected in the variance of the weights and, therefore, in the statistical uncertainty of the integral estimate, when considering a fixed number of integrand evaluations.

We first discuss the canonical approach, in which the integrators are trained on the absolute value of the integrand and the entire phase space is integrated together. For integrands of non-definite sign, if no positive/negative splitting is performed, there is a lower bound on the uncertainty, denoted as δ_{opt} , based on the positive-to-negative ratio, see Eq. (2.7). Suppose the adaptation of the integrator, i.e. its

sampling distribution, to the integrand is expressive enough and there is a sufficiently balanced positive-to-negative ratio. In that case, this effect will dominate the estimated uncertainty, and there is no improvement to be expected beyond that point by the sheer expressiveness of the integrator. In the case of a negligible negative (or positive, for that matter) contribution, as for instance in the Born cross section σ^B arising from negative PDF values, however, the reduction of the variance of the positive (or negative) part dominates. The expected enhanced expressiveness of the Normalizing Flow-based methods can be seen in the corresponding uncertainty estimates. For the Born contribution, the VEGAS integrator results in a cross-section estimate of $\hat{\sigma}_{\text{VEGAS}}^B = 532.8 \pm 0.2$ pb, while for the flow samplers we obtain $\hat{\sigma}_{\text{ODE}}^B = 532.79 \pm 0.05$ pb and $\hat{\sigma}_{\text{CL}}^B = 532.80 \pm 0.03$ pb, respectively. Accordingly, the uncertainty gets reduced by a factor 4 for the ODE flow and a factor 7 for the CL flow, corresponding to a performance gain of a factor 16 and 49, respectively. However, the formal bound on the uncertainty of $\delta_{\text{opt}} = 0.01$ pb is not yet reached. A smaller but still visible performance gain can be observed in more complicated contributions with subtractions like σ^{RU} , σ^{RRF} , σ^{RRSU} and σ^{RVF} , where we find a reduction of the uncertainties by a factor of 2 with respect to the VEGAS method if a flow-based integrator is used. The performances of the ODE and CL samplers are thereby largely on par. Importantly, we can observe that both integrators in these cases saturate or are close to saturating the corresponding uncertainty bound δ_{opt} . Further, we find that for all the other (simpler) NLO and NNLO QCD contributions (σ^i for $i \in \{\text{RF}, \text{VF}, \text{RVFR}, \text{RV DU}, \text{VVF}, \text{RRDU}\}$), this bound can be saturated by all integrators. For example, for $\hat{\sigma}^{\text{RF}}$ the bound evaluates to $\delta_{\text{opt}} = 0.8$ pb, and we find uncertainties for the three samplers given by $\delta\hat{\sigma}_{\text{VEGAS}}^{\text{RF}} = 1$ pb, and $\delta\hat{\sigma}_{\text{ODE}}^{\text{RF}} = \delta\hat{\sigma}_{\text{CL}}^{\text{RF}} = 0.9$ pb, respectively.

Besides the uncertainty of the integral estimate, the actual weight distribution can give insights into how well an integrator learned the target distribution. For σ^{RF} and σ^{RRF} , we show the weight distribution for the absolute integrand as well as the distributions of the positive and negative weights separately in Fig. 2. First, it can be noticed that the distributions of the ODE and CL integrators are much more narrow than the VEGAS ones, and second, we can see that the maximum weights are reduced, i.e. there are significantly less events with large weights in the ODE and CL samples. The distributions suggest that the performance gain using the flow-based integrators should be significant. As mentioned, the only mild performance gain in these cases is caused by sizable cancellations between the positive and negative contributions.

The reduction in the weight variance can be traced back to non-factorizable features in the phase space that the flow-based methods can learn, but the VEGAS model can not. A visualization of such a feature is given in Fig. 3 for the case of the σ^{RRF} contribution. We show a 2D histogram (the counts are indicated through the heat map), with the coordinate axes given by two integration parameters, for

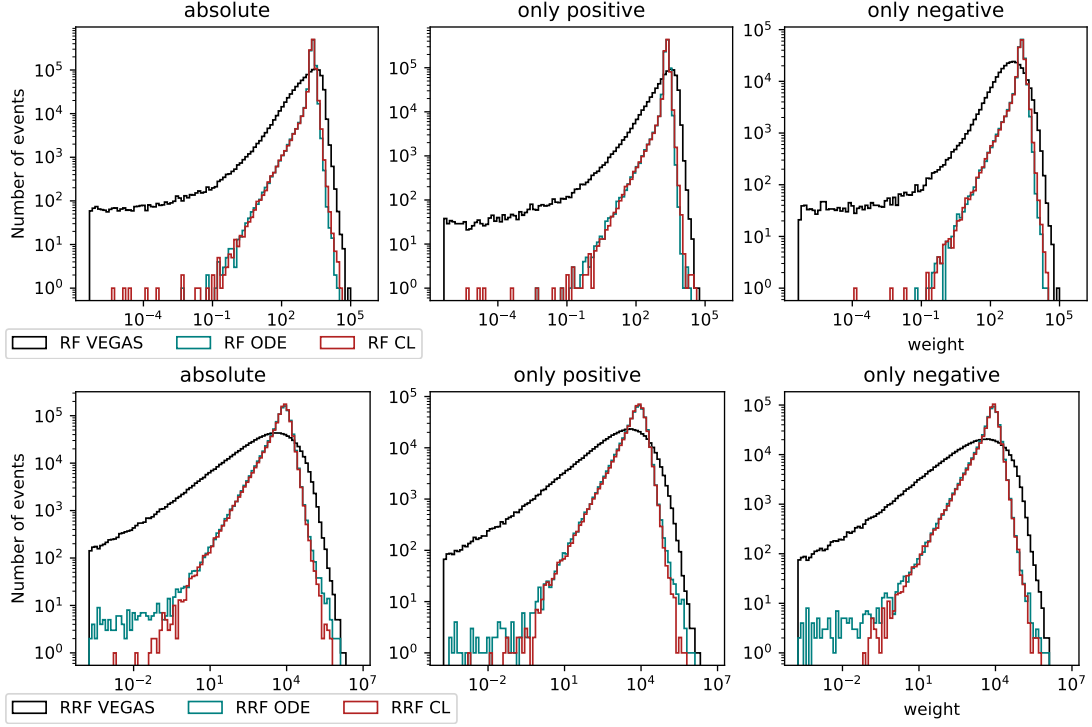


Figure 2. The distribution of weights for the σ^{RF} (top row) and σ^{RRF} contributions (bottom row). The first column shows the absolute weight, and the second and third columns show the positive and negative weight distributions. The distributions are shown for the VEGAS, CL and ODE integrators, all trained on the absolute integrand.

the VEGAS integrator and the CL integrator (the ODE one looks identical to the CL one). The non-trivial structure learned by the CL integrator can be seen by eye, and it is clear that VEGAS learned a different distribution.

Integral-variance reduction: training on split integrand

The advantage of using a flow-based method is clearly diminished when there are large cancellations between positive and negative contributions, and the gain based on variance reduction is thus limited. This implies that further improvement on the quality of the integrator, through more parameters or training data, will not lead to significant performance improvements. The picture changes, however, when a positive/negative split integrand is considered and the positive and negative regions are treated as independent integrals. In this case, there is no lower bound on the variance upon integration, i.e. a vanishing uncertainty can in principle be achieved. Therefore, the improved adaptability of the flows can be fully exploited. However, the integration problem is now more challenging because the efficiency of hitting the right phase-space region becomes crucial for reaching a small variance (missed points are counted as zero weight, increasing the variance). The phase-space boundaries between the positive and negative regions are non-trivial and likely non-factorizable.

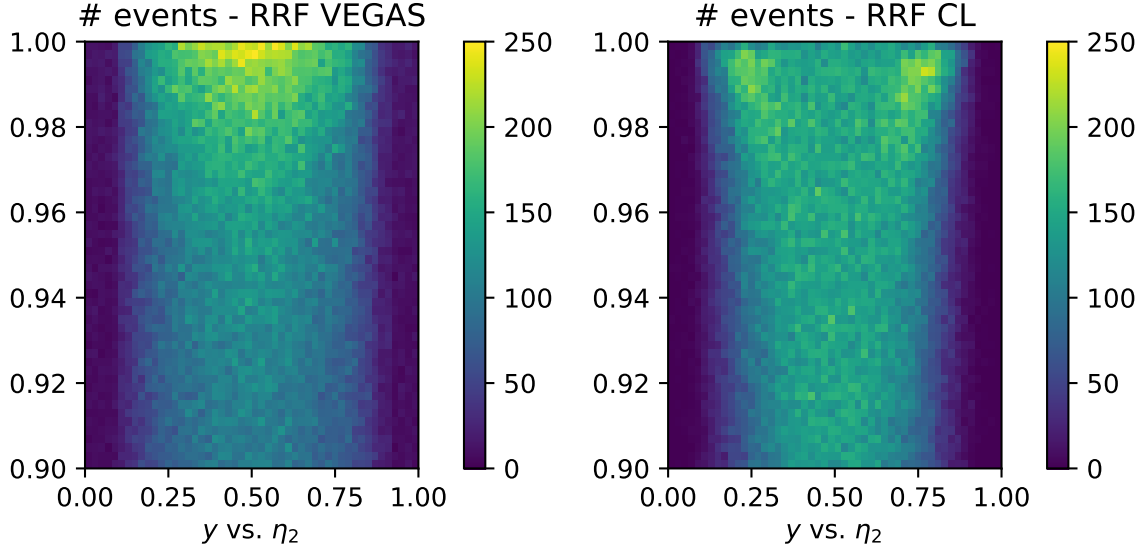


Figure 3. Example of non-factorizable correlations between two phase-space parameters for the σ^{RRF} contribution. The left panel shows the learned sampling distribution of VEGAS, and the right panel that of the CL integrator.

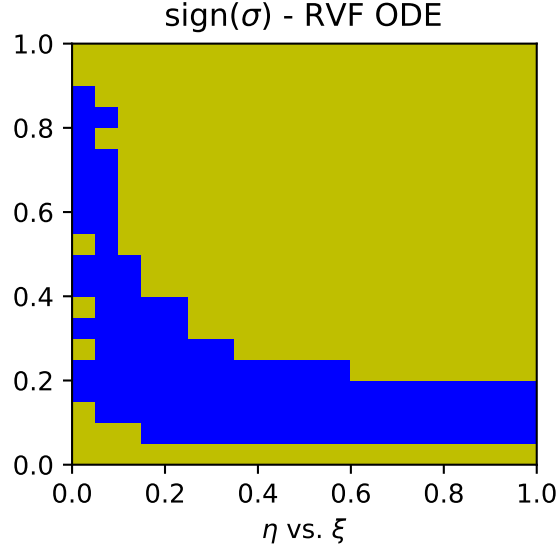


Figure 4. The sign (positive - yellow, negative - blue) of the binned cross section of the σ^{RVF} contribution as a function of two input parameters is shown. All other dimensions are integrated out.

In Fig. 4 we show the sign of the integral, differential and binned in two integration variables, for the σ^{RVF} contribution, as an example for such non-trivial boundaries. This is only a two-dimensional projection and in reality the boundaries between the signs have a complicated shape in multidimensional space. Naturally, we expect the VEGAS algorithm to have trouble performing well in this split integrand case.

The results for the positive and negative cross-section contributions $\hat{\sigma}^\pm$ individually are given in Tabs. 1, 2 and 3, together with their combination $\hat{\Sigma}^\pm$. The correctness of this approach is validated by the agreement of $\hat{\Sigma}^\pm$ with the canonical approach, i.e. the respective $\hat{\sigma}$, within the estimated statistical uncertainty. To gauge the performance gain of the positive/negative splitting, the uncertainty of this quantity has to be compared to $\delta\hat{\sigma}$. First, a general observation: we find an improved or at least comparable performance for all integrators compared to the traditional non-split integrands with the same integrator. We can sometimes observe significant performance gains when using the flow-based integrators. A relevant example at NLO is the σ^{RF} contribution. Both, for the ODE and the CL sampler, the uncertainty drops by a factor of 3 from $\delta\hat{\sigma}^{\text{RF}} = 0.9 \text{ pb}$, obtained when training on the absolute integrand, to $\delta\hat{\Sigma}^{\text{RF}} = 0.3 \text{ pb}$ based on the split integrand. This corresponds to a factor of 9 in performance gain. In fact, both samplers manage to clearly go below the uncertainty limit of $\delta_{\text{opt}} = 0.8 \text{ pb}$ when training on the absolute integrand. Such improvement is absent for the VEGAS integrator, what can be traced back to its poor phase-space efficiency $\epsilon_\Phi^\pm$, which only reaches 73% and 34% for the positive and negative phase-space region, respectively. The flow-based methods instead reach efficiencies of about 99% in this case. At NNLO QCD, the most impressive case is σ^{RVF} , which features the highest CPU evaluation costs per phase-space point. For the RVF contribution the ODE sampler reduces the uncertainty by a factor 3 upon positive/negative splitting. However, the rather challenging RRF contribution does not benefit from the splitting, given the training setup, as sampling efficiencies $\epsilon_\Phi^\pm$ of only 85% are reached.

Eventually, not only the achieved gain for each cross-section contribution is relevant, but it is also crucial to factor in the corresponding computational costs and the relative contribution to the final result. To this end, the amount of phase-space points evaluated for each individual contribution needs to be optimized. This can be realized for a requested target uncertainty given by

$$\delta\sigma_{\text{target}} = \delta\hat{\sigma}_{\text{tot}}(\mathbf{n}) = \sqrt{\sum_i (\delta\hat{\sigma}^i(n_i))^2} = \sqrt{\sum_i \frac{\text{Var}(\hat{\sigma}^i)}{n_i - 1}}, \quad (4.2)$$

where the sum i runs over all contributions included, $\text{Var}(\hat{\sigma}^i)$ denotes the corresponding weight variance, and the elements n_i of vector $\mathbf{n}$ represent the number of points in each contribution. Under this constraint the computational costs C , defined by

$$C = \sum_i C_i n_i, \quad (4.3)$$

can then be minimized, with C_i the costs per phase-space point for contribution i . This minimization problem can be solved numerically using Lagrange multipliers. Folding the costs and contributions to the final result in this way, we are left with a

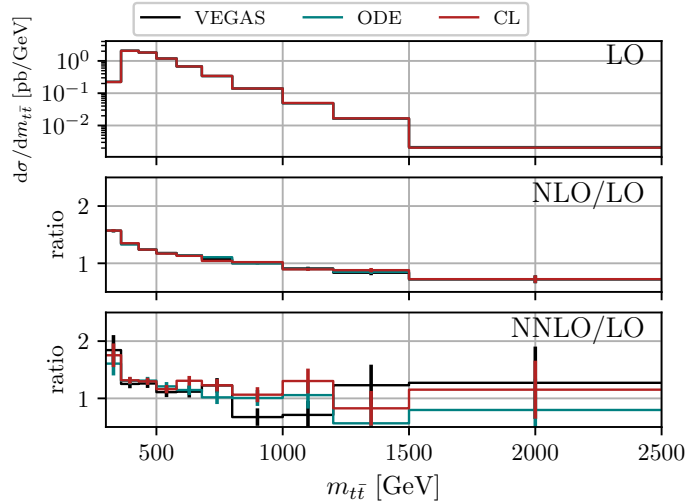


Figure 5. Differential cross section with respect to the top-pair invariant mass $m_{t\bar{t}}$. The upper panel shows the absolute LO QCD result obtained with the VEGAS, ODE and CL integrator. The middle and lower panels show the NLO and NNLO QCD results as a ratio to the LO result, respectively. The vertical bars indicate the statistical uncertainty.

gain in computational efficiency of a factor ≈ 2 for the flow-based methods in the case of training on the absolute integrand and a factor ≈ 8 in the case of positive/negative separated integrands with respect to our VEGAS baseline. At NNLO QCD accuracy, the calculation is dominated ($\approx 80\%$) by the computation of the double-real contributions, followed by the real-virtual part. Note, the above cost estimate does not include the GPU time for training and inference of the phase-space sampler. This implies that the quoted gains present an upper bound, reached in the limit where the matrix-element evaluation time clearly dominates.

For reference, for the *RRF* contribution (which has the largest flows and hence NNs) we found that we need 3 (6) minutes for one training iteration and 1 (3) minutes for the generation step on one half of a Nvidia A100 chip for the CL(ODE)-flow. These numbers imply a total training and generation time for this case of approximately 30 to 90 minutes for the entire training sequence. These are to be compared with approximately 10 hours of integrand evaluation time on a CPU, totalling 10 million evaluations. However, the GPU numbers should be taken with a grain of salt, as I/O operations contribute significantly due to the short runtime.

Differential cross sections

Up to here, we focused on the total cross section, which is also the central training objective for our integrators. However, we want to explicitly stress that we can simultaneously compute arbitrary differential cross sections for infrared and collinear safe observables with our methods. As an example, in Fig. 5 we show the differential distribution with respect to the invariant mass of the top-quark pair at LO,

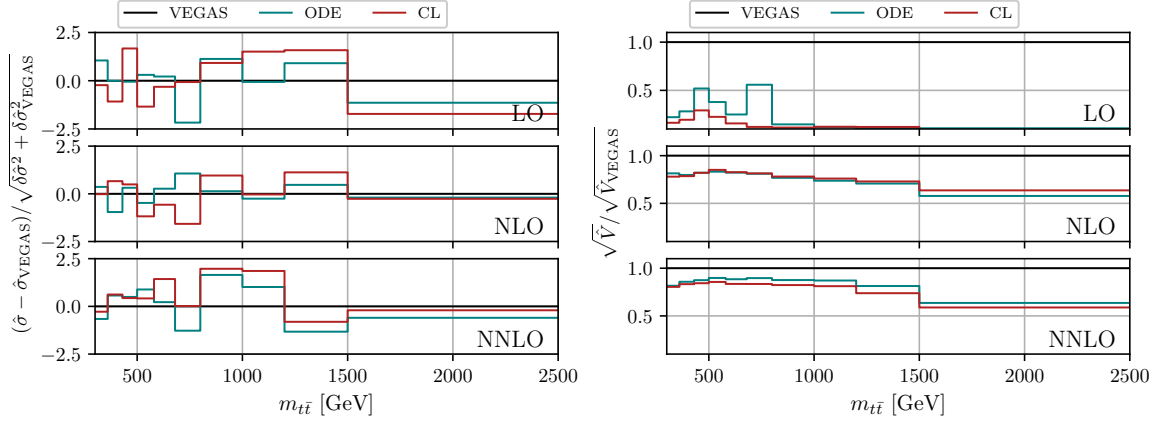


Figure 6. The left-hand column shows the agreement between VEGAS, ODE and CL as a function of the top-pair invariant mass $m_{t\bar{t}}$. The right-hand side shows the variance of the weights in each bin as a ratio to the VEGAS variance.

as obtained with the three considered integration algorithms. The corresponding NLO and NNLO results are presented as ratios to the LO. Notably, the three integrators faithfully reproduce the differential distributions, and the different results agree within their statistical uncertainty in each bin, which is highlighted on the left-hand side of Fig. 6, showing the deviation relative to the combined statistical uncertainty. Further, on the right-hand side in Fig. 6, we can also demonstrate that the flow-based integrators reduce the weight variance in each bin with respect to the VEGAS method.

Unweighted event generation

The capability to reduce the variance of the sample weights is not the only figure of merit that can be considered when comparing importance-sampling methods. A different metric is provided by the unweighting efficiency, given by $\epsilon = \langle w \rangle / w_{\max}$. Accordingly, the unweighting efficiency depends on the weight maximum and is thus rather sensitive to large-weight outliers. In Tabs. 1, 2 and 3 we quote the achieved unweighting efficiencies for the positive and negative region of phase space as ϵ^+ and ϵ^- separately. In addition, we provide the efficiency estimate when considering a reduced maximum weight. This reduced maximum weight is determined such that the largest weights contributing cumulatively less than 0.1% to the total cross section are discarded, cf. Sec. 2.1 for details. This quantity, denoted as $\epsilon_{0.1\%}^{\pm}$, is less sensitive to weight fluctuations within a fixed sample and leads to higher efficiencies by design. We want to note that if the largest-weight event corresponds to a substantial fluctuation, i.e. a single event contributing more than 0.1% to the total integral, then no improvement can be achieved with this prescription. The improvements in terms of computational costs are linearly correlated with the efficiency.

We can observe increases in the unweighting efficiency, specifically in the $\epsilon_{0.1\%}^{\pm}$ measure, using the flow-based integrators in various contributions. The unweighting efficiency for contributions with tree-level kinematics, such as σ^B , σ^{VF} or σ^{VVF} , improve by a factor of 2 to 4 with respect to VEGAS when using the flow-based approaches. Further, we see similar improvements for single-radiation contributions. For example, σ^{RF} and σ^{RVF} , see an improvement by a factor of 3 in unweighting efficiency using the ODE flow compared to VEGAS. However, for σ^{RVF} the CL sampler yields a rather low efficiency of $\epsilon_{CL}^+ = 7.6 \cdot 10^{-4}$, that is dominated by a large-weight outlier, given that $\epsilon_{0.1\%,CL}^+$ has the same value. For the ODE sampler we find for the nominal maxima efficiencies of $\epsilon_{ODE}^+ = 2.9 \cdot 10^{-3}$ and $\epsilon_{ODE}^- = 5.1 \cdot 10^{-2}$, compared to the VEGAS values of $\epsilon_{VEGAS}^+ = 1.3 \cdot 10^{-2}$ and $\epsilon_{VEGAS}^- = 1.4 \cdot 10^{-2}$. After reducing the maxima, however, the ODE sampler outperforms VEGAS for both regions, where $\epsilon_{0.1\%,ODE}^+ = 6.7 \cdot 10^{-2}$ vs. $\epsilon_{0.1\%,VEGAS}^+ = 2.4 \cdot 10^{-2}$ and $\epsilon_{0.1\%,ODE}^- = 2.5 \cdot 10^{-1}$ vs. $\epsilon_{0.1\%,VEGAS}^- = 3.8 \cdot 10^{-2}$. For the double-real contributions, i.e. σ^{RRF} , σ^{RRSU} and σ^{RRDU} , we do not find significant gains with respect to VEGAS.

These observations can be correlated with the weight distributions shown in Fig. 2. We see that while the flow-based methods feature visibly sharper peaks, both, for the positive and negative parts, the maximum weights (i.e. the ends of the tail to the right) are similar to the VEGAS results, in particular for the σ^{RRF} contribution. While the outlined prescription for reducing the maximum weight does lead to similar performances of VEGAS and the flow-based models, it is clear that the number of events with overweights will be substantially larger in the VEGAS case. This results in a smaller effective sample size compared to the samples generated with the flow methods.

While STRIPPER is not an event generator, (partially) unweighted events can be used to store a calculation on disk for further analysis in the HIGHTEA framework [90]. The gains in unweighting efficiency directly translate into reduced computational costs for the event generation.

5 Conclusions

In this work we studied the application of two Normalizing-Flow integrators for the evaluation of NNLO QCD cross sections. In particular, we considered Coupling Layer and Continuous Normalizing Flows and applied them to top-quark pair production in the gluonic channel. The NNLO QCD calculation follows the STRIPPER approach for the subtraction of soft and collinear singularities. In order to reach NNLO QCD accuracy, besides the lowest-order Born contribution, various (subtracted) single- and double real-emission and one- and two-loop virtual corrections, as well as interference terms need to be evaluated, and several contributing partonic channels have to be considered. The STRIPPER subtraction formalism is based on selector functions that address specific soft and collinear limits and, consequently, sectorize phase space. In

turn, this results in a significant proliferation of phase-space integrals that need to be performed. Furthermore, the integrands of the various contributions are often not of definite sign, what limits the performance of any integration method whose sampling distribution is adapted to the absolute value of the integrand, as is canonically done. To address this issue, we consider the stratification of the integrands into their positive and negative parts and separately train the sampling algorithms. Accordingly, for an optimal sampling distribution the integral variance would vanish. However, for this approach to be successful the sampler has to efficiently predict the sign of the integrand for a given phase-space point. Furthermore, we developed the means to condition the flow-based samplers on other discrete variables, including a sector index and helicity states.

As main figure of merit for our novel sampling methods we consider the statistical uncertainty of the obtained cross-section estimates. Furthermore, we monitor the resulting weight distributions and from these extract unweighting efficiencies for performing a rejection sampling to obtain unit-weight events. To this end, we studied the impact of a systematic reduction of the maximum weights, corresponding to the production of partially unweighted events, but with potentially increased efficiency. We compare all our results against the VEGAS sampler, that closely reproduces the performance of the current default method used in STRIPPER, namely the PARNI algorithm [102].

The enhanced expressiveness and adaptability of the flow-based samplers to the target distribution with respect to the factorized density learned by VEGAS results in significant reductions of the weight-distribution variances. In particular when splitting the integrands into their positive and negative contributions did we find large performance gains for both neural importance samplers. Overall this can be summarized in the computational costs for obtaining an estimate for the total cross section at predefined statistical uncertainty. When dominated by the time needed to evaluate the integrands, the costs can be reduced by a factor of 2 when training the samplers on the absolute integrand, and a factor of 8 when splitting the integrands into their positive and negative parts. Besides for the total cross sections, the flow-based samplers also yield reduced weight variances in binned differential distributions, as exemplified for the top-pair invariant mass observable. For the unweighting efficiencies of the various cross-section contributions we also find improvements when using flow-based samplers, although rare large-weight outliers can sometimes obscure this picture.

An improved unweighting efficiency is rather a byproduct of the variance-reduction training incentive of importance samplers. Large-weight outliers, that impair the unweighting efficiency, will often originate from rarely sampled phase-space regions, where the samplers have not seen a lot of training data. To improve on that, the phase-space distribution could be biased towards overproducing such rare configurations. This procedure could also be employed to reduce the statistical uncertainty

in the tails of differential distributions.

In summary, the application of modern flow-based sampling methods provides large potential to significantly reduce the computational costs of cutting edge higher-order calculations. We have worked out a rather simple workflow for training and application of the samplers, showcased for top-quark pair production at NNLO QCD with STRIPPER, that could similarly be used for other calculational frameworks.

Acknowledgements

We acknowledge fruitful discussions with David Yallup at an early stage of the project. TJ would like to thank Bernhard Schmitzer and Fabian Sinz for insightful discussions about Continuous Normalizing Flows and Flow Matching. TJ and SS are grateful for financial support from BMBF (projects 05D23MG1 and 05H24MGA) and funding by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) through project 456104544. TJ expresses his gratitude to CIDAS for supporting him with a fellowship.

The authors gratefully acknowledge the computing time granted by the Resource Allocation Board and provided on the supercomputer Emmy/Grete at NHR-Nord@Göttingen as part of the NHR infrastructure. The calculations for this research were conducted with computing resources under the project `nhr_ni_starter_22045`.

RP acknowledges the support of Narodowe Centrum Nauki under Grant No. 2023/49/B/ST2/04330 (SNAIL). The authors acknowledge support from the COMETA COST Action CA22130.

A Numerical results

Contribution σ^i		$\sigma^{\text{B}} \cdot 10^{-2}$	$\sigma^{\text{RF}} \cdot 10^{-2}$	$\sigma^{\text{RU}} \cdot 10^{-2}$	$\sigma^{\text{VF}} \cdot 10^{-1}$				
δ_{opt} [pb]		0.0001	0.008	0.01	0.006				
cost C_i [a.u.]		1	6	2	1				
$\hat{\sigma} \pm \delta\hat{\sigma}$ [pb]	VEGAS	5.328(2)	6.97(1)	−5.38(2)	− 4.487(6)				
	ODE	5.3279(5)	6.980(9)	− 5.39(1)	− 4.483(6)				
	CL	5.3280(3)	6.972(9)	− 5.39(1)	− 4.492(6)				
$\hat{\sigma}^+ \pm \delta\hat{\sigma}^+$ [pb]	VEGAS	5.328(2)	8.96(1)	4.432(6)	1.4030(8)				
	ODE	5.3287(4)	8.949(2)	4.419(2)	1.4018(1)				
	CL	5.3281(3)	8.949(2)	4.410(2)	1.40155(9)				
$\hat{\sigma}^- \pm \delta\hat{\sigma}^-$ [pb]	VEGAS	−6.446(5) $\times 10^{-4}$	−1.981(4)	−9.810(8)	−5.894(3)				
	ODE	−6.450(1) $\times 10^{-4}$	− 1.9844(6)	−9.802(3)	−5.8932(5)				
	CL	− 6.4488(3) $\times 10^{-4}$	−1.9821(7)	− 9.800(2)	− 5.8918(3)				
$\hat{\Sigma}^\pm \pm \delta\hat{\Sigma}^\pm$ [pb]	VEGAS	5.328(2)	6.97(1)	−5.38(1)	−4.491(3)				
	ODE	5.3281(4)	6.965(3)	− 5.381(4)	−4.4914(6)				
	CL	5.3274(3)	6.967(3)	− 5.390(4)	− 4.4903(4)				
ϵ_Φ^+	VEGAS	0.99	0.73	0.64	0.81				
	ODE	1.00	0.99	0.99	1.00				
	CL	1.00	0.99	0.99	1.00				
ϵ_Φ^-	VEGAS	0.83	0.38	0.85	0.95				
	ODE	1.00	0.99	0.99	1.00				
	CL	1.00	0.99	0.99	1.00				
$\epsilon^+ \mid \epsilon_{0.1\%}^+$	VEGAS	0.13	0.43	0.048	0.098	0.020	0.048	0.082	0.21
	ODE	0.33	0.70	0.025	0.30	0.0059	0.099	0.11	0.56
	CL	0.53	0.81	0.028	0.24	0.0082	0.046	0.17	0.63
$\epsilon^- \mid \epsilon_{0.1\%}^-$	VEGAS	0.016	0.066	0.013	0.021	0.049	0.17	0.12	0.27
	ODE	0.055	0.36	0.028	0.17	0.020	0.16	0.12	0.73
	CL	0.11	0.79	0.0074	0.060	0.0090	0.19	0.40	0.79

Table 1. Results obtained with the VEGAS, ODE and CL integrators for the Born and NLO QCD contributions to gluonic top-quark pair production. The training of all samplers is based on 10^7 points (10×10^6 points with intermediate adaptation). The quoted results are based on 10^6 integrand evaluations with frozen integrator settings. Note, for better readability all cross-section values have been rescaled by factors indicated in the first row, while all efficiencies are quoted with their nominal value.

Contribution σ^i		$\sigma^{\text{RRF}} \cdot 10^{-2}$	$\sigma^{\text{RRSU}} \cdot 10^{-3}$			$\sigma^{\text{RRDU}} \cdot 10^{-3}$	
δ_{opt} [pb]		0.008	0.004			0.01	
cost C_i [a.u.]		53	26			8	
$\hat{\sigma} \pm \delta\hat{\sigma}$ [pb]	VEGAS	$-0.30(2)$	$2.835(7)$			$-2.46(2)$	
	ODE	$-\mathbf{0.33(1)}$	$2.823(5)$			$-2.42(2)$	
	CL	$-\mathbf{0.32(1)}$	$\mathbf{2.829(4)}$			$-\mathbf{2.41(1)}$	
$\hat{\sigma}^+ \pm \delta\hat{\sigma}^+$ [pb]	VEGAS	$3.82(1)$	$3.718(6)$			$5.639(7)$	
	ODE	$3.812(6)$	$\mathbf{3.706(3)}$			$\mathbf{5.644(4)}$	
	CL	$\mathbf{3.814(5)}$	$\mathbf{3.704(3)}$			$\mathbf{5.643(4)}$	
$\hat{\sigma}^- \pm \delta\hat{\sigma}^-$ [pb]	VEGAS	$-4.14(1)$	$-\mathbf{0.881(2)}$			$-8.078(9)$	
	ODE	$-\mathbf{4.142(6)}$	$-0.885(5)$			$-8.071(7)$	
	CL	$-\mathbf{4.134(6)}$	$-\mathbf{0.885(2)}$			$-\mathbf{8.076(5)}$	
$\hat{\Sigma}^\pm \pm \delta\hat{\Sigma}^\pm$ [pb]	VEGAS	$-0.31(3)$	$2.836(8)$			$-2.44(2)$	
	ODE	$-\mathbf{0.33(1)}$	$2.821(7)$			$-2.43(1)$	
	CL	$-\mathbf{0.32(1)}$	$\mathbf{2.819(4)}$			$-\mathbf{2.433(9)}$	
ϵ_Φ^+	VEGAS	0.54	0.73			0.80	
	ODE	0.84	$\mathbf{0.96}$			$\mathbf{0.99}$	
	CL	$\mathbf{0.86}$	0.95			0.98	
ϵ_Φ^-	VEGAS	0.51	0.40			0.80	
	ODE	$\mathbf{0.85}$	0.83			0.98	
	CL	$\mathbf{0.85}$	$\mathbf{0.84}$			$\mathbf{0.99}$	
$\epsilon^+ \mid \epsilon_{0.1\%}^+$	VEGAS	$\mathbf{0.0040}$	0.0052	0.0073	0.017	0.0018	0.022
	ODE	0.0013	0.0027	0.0028	$\mathbf{0.022}$	$\mathbf{0.0030}$	$\mathbf{0.023}$
	CL	0.0022	$\mathbf{0.0056}$	$\mathbf{0.0096}$	$\mathbf{0.022}$	0.0029	0.011
$\epsilon^- \mid \epsilon_{0.1\%}^-$	VEGAS	$\mathbf{0.0041}$	0.0053	$\mathbf{0.0035}$	$\mathbf{0.010}$	0.0037	$\mathbf{0.019}$
	ODE	0.0019	0.0053	0.000 25	0.000 25	0.0018	0.0096
	CL	0.0034	$\mathbf{0.0060}$	0.0012	0.0028	$\mathbf{0.0044}$	0.011

Table 2. Results obtained with the VEGAS, ODE and CL integrators for the double-real NNLO QCD contributions to gluonic top-quark pair production. The training of all samplers is based on 10^7 points (10×10^6 points with intermediate adaptation). The quoted results are based on 10^6 integrand evaluations with frozen integrator settings. Note, for better readability all cross-section values have been rescaled by factors indicated in the first line, while all efficiencies are quoted with their nominal value.

Contribution σ^i		$\sigma^{\text{RVF}} \cdot 10^{-2}$		$\sigma^{\text{RVFR}} \cdot 10^{-2}$		$\sigma^{\text{RVDU}} \cdot 10^{-3}$		$\sigma^{\text{VVF}} \cdot 1$	
δ_{opt} [pb]		0.008		0.002		0.005		0.01	
cost C_i [a.u.]		1542		3		3		13	
$\hat{\sigma} \pm \delta\hat{\sigma}$ [pb]	VEGAS	−6.81(2)		0.286(2)		0.267(5)		14.77(1)	
	ODE	− 6.784(8)		0.289(2)		0.271(5)		14.76(1)	
	CL	− 6.778(8)		0.285(2)		0.276(5)		14.77(1)	
$\hat{\sigma}^+ \pm \delta\hat{\sigma}^+$ [pb]	VEGAS	1.669(3)		1.051(1)		2.381(1)		16.711(7)	
	ODE	1.6764(9)		1.0495(4)		2.3806(4)		16.720(3)	
	CL	1.676(2)		1.0500(6)		2.3799(4)		16.718(2)	
$\hat{\sigma}^- \pm \delta\hat{\sigma}^-$ [pb]	VEGAS	−8.454(7)		−0.764(1)		−2.111(1)		−1.955(1)	
	ODE	− 8.458(2)		− 0.7627(3)		−2.1109(5)		−1.9554(2)	
	CL	−8.456(3)		−0.7626(6)		− 2.1091(4)		− 1.9548(1)	
$\hat{\Sigma}^\pm \pm \delta\hat{\Sigma}^\pm$ [pb]	VEGAS	−6.785(9)		0.287(3)		0.270(2)		14.756(8)	
	ODE	− 6.781(3)		0.2868(6)		0.2697(9)		14.765(3)	
	CL	−6.780(5)		0.287(1)		0.2708(8)		14.763(2)	
ϵ_Φ^+	VEGAS	0.55		0.59		0.95		0.96	
	ODE	0.99		0.99		1.00		1.00	
	CL	0.99		0.98		1.00		1.00	
ϵ_Φ^-	VEGAS	0.87		0.50		0.92		0.82	
	ODE	0.99		0.98		1.00		1.00	
	CL	0.99		0.99		1.00		1.00	
$\epsilon^+ \mid \epsilon_{0.1\%}^+$	VEGAS	0.013	0.024	0.025	0.060	0.029	0.22	0.015	0.10
	ODE	0.0029	0.067	0.0081	0.099	0.030	0.27	0.0086	0.49
	CL	0.000 76	0.000 76	0.0024	0.035	0.010	0.22	0.020	0.31
$\epsilon^- \mid \epsilon_{0.1\%}^-$	VEGAS	0.014	0.038	0.021	0.050	0.011	0.11	0.024	0.17
	ODE	0.051	0.25	0.017	0.095	0.017	0.18	0.17	0.49
	CL	0.0054	0.13	0.0017	0.018	0.022	0.22	0.10	0.68

Table 3. Results obtained with the VEGAS, ODE and CL integrators for the virtual NNLO QCD contributions to gluonic top-quark pair production. The training of all samplers is based on 10^7 points (10×10^6 points with intermediate adaptation). The quoted results are based on 10^6 integrand evaluations with frozen integrator settings. Note, for better readability all cross-section values have been rescaled by factors indicated in the first line, while all efficiencies are quoted with their nominal value.

References

- [1] A. Kanaki and C. G. Papadopoulos, *HELAC: A Package to compute electroweak helicity amplitudes*, *Comput. Phys. Commun.* **132** (2000) 306–315, [[hep-ph/0002082](#)].
- [2] F. Maltoni and T. Stelzer, *MadEvent: Automatic event generation with MadGraph*, *JHEP* **02** (2003) 027, [[hep-ph/0208156](#)].
- [3] F. Krauss, R. Kuhn, and G. Soff, *AMEGIC++ 1.0: A Matrix element generator in C++*, *JHEP* **02** (2002) 044, [[hep-ph/0109036](#)].
- [4] W. Kilian, T. Ohl, and J. Reuter, *WHIZARD: Simulating Multi-Particle Processes at LHC and ILC*, *Eur. Phys. J. C* **71** (2011) 1742, [[arXiv:0708.4233](#)].
- [5] T. Gleisberg and S. Hoeche, *Comix, a new matrix element generator*, *JHEP* **12** (2008) 039, [[arXiv:0808.3674](#)].
- [6] F. Cascioli, P. Maierhofer, and S. Pozzorini, *Scattering Amplitudes with Open Loops*, *Phys. Rev. Lett.* **108** (2012) 111601, [[arXiv:1111.5206](#)].
- [7] F. Buccioni, et al., *OpenLoops 2*, *Eur. Phys. J. C* **79** (2019) 866, [[arXiv:1907.13071](#)].
- [8] S. Actis, et al., *RECOLA: REcursive Computation of One-Loop Amplitudes*, *Comput. Phys. Commun.* **214** (2017) 140–173, [[arXiv:1605.01090](#)].
- [9] A. Denner, J.-N. Lang, and S. Uccirati, *Recola2: REcursive Computation of One-Loop Amplitudes 2*, *Comput. Phys. Commun.* **224** (2018) 346–361, [[arXiv:1711.07388](#)].
- [10] V. Hirschi, et al., *Automation of one-loop QCD corrections*, *JHEP* **05** (2011) 044, [[arXiv:1103.0621](#)].
- [11] S. Frixione, Z. Kunszt, and A. Signer, *Three jet cross-sections to next-to-leading order*, *Nucl. Phys. B* **467** (1996) 399–442, [[hep-ph/9512328](#)].
- [12] S. Catani and M. H. Seymour, *A General algorithm for calculating jet cross-sections in NLO QCD*, *Nucl. Phys. B* **485** (1997) 291–419, [[hep-ph/9605323](#)]. [Erratum: *Nucl.Phys.B* 510, 503–504 (1998)].
- [13] S. Catani, S. Dittmaier, M. H. Seymour, and Z. Trocsanyi, *The Dipole formalism for next-to-leading order QCD calculations with massive partons*, *Nucl. Phys. B* **627** (2002) 189–265, [[hep-ph/0201036](#)].
- [14] H. A. Chawdhry, M. L. Czakon, A. Mitov, and R. Poncelet, *NNLO QCD corrections to three-photon production at the LHC*, *JHEP* **02** (2020) 057, [[arXiv:1911.00479](#)].
- [15] S. Kallweit, V. Sotnikov, and M. Wiesemann, *Triphoton production at hadron colliders in NNLO QCD*, *Phys. Lett. B* **812** (2021) 136013, [[arXiv:2010.04681](#)].

- [16] H. A. Chawdhry, M. Czakon, A. Mitov, and R. Poncelet, *NNLO QCD corrections to diphoton production with an additional jet at the LHC*, *JHEP* **09** (2021) 093, [[arXiv:2105.06940](#)].
- [17] M. Czakon, A. Mitov, and R. Poncelet, *Next-to-Next-to-Leading Order Study of Three-Jet Production at the LHC*, *Phys. Rev. Lett.* **127** (2021) 152001, [[arXiv:2106.05331](#)]. [Erratum: *Phys.Rev.Lett.* 129, 119901 (2022)].
- [18] X. Chen, T. Gehrmann, E. W. N. Glover, A. Huss, and M. Marcoli, *Automation of antenna subtraction in colour space: gluonic processes*, *JHEP* **10** (2022) 099, [[arXiv:2203.13531](#)].
- [19] H. B. Hartanto, R. Poncelet, A. Popescu, and S. Zoia, *Next-to-next-to-leading order QCD corrections to $Wb\bar{b}$ production at the LHC*, *Phys. Rev. D* **106** (2022) 074016, [[arXiv:2205.01687](#)].
- [20] S. Badger, et al., *Isolated photon production in association with a jet pair through next-to-next-to-leading order in QCD*, *JHEP* **10** (2023) 071, [[arXiv:2304.06682](#)].
- [21] F. Buccioni, et al., *Precise Predictions for Event Shapes in Diphoton Production at the LHC*, *Phys. Rev. Lett.* **134** (2025) 171901, [[arXiv:2501.14021](#)].
- [22] S. Catani and M. Grazzini, *An NNLO subtraction formalism in hadron collisions and its application to Higgs boson production at the LHC*, *Phys. Rev. Lett.* **98** (2007) 222002, [[hep-ph/0703012](#)].
- [23] R. Bonciani, S. Catani, M. Grazzini, H. Sargsyan, and A. Torre, *The q_T subtraction method for top quark production at hadron colliders*, *Eur. Phys. J. C* **75** (2015) 581, [[arXiv:1508.03585](#)].
- [24] J. Gaunt, M. Stahlhofen, F. J. Tackmann, and J. R. Walsh, *N -jettiness Subtractions for NNLO QCD Calculations*, *JHEP* **09** (2015) 058, [[arXiv:1505.04794](#)].
- [25] R. Boughezal, C. Focke, X. Liu, and F. Petriello, *W -boson production in association with a jet at next-to-next-to-leading order in perturbative QCD*, *Phys. Rev. Lett.* **115** (2015) 062002, [[arXiv:1504.02131](#)].
- [26] R. Boughezal, X. Liu, and F. Petriello, *Power Corrections in the N -jettiness Subtraction Scheme*, *JHEP* **03** (2017) 160, [[arXiv:1612.02911](#)].
- [27] I. Moulton, L. Rothen, I. W. Stewart, F. J. Tackmann, and H. X. Zhu, *Subleading Power Corrections for N -Jettiness Subtractions*, *Phys. Rev. D* **95** (2017) 074023, [[arXiv:1612.00450](#)].
- [28] I. Moulton, L. Rothen, I. W. Stewart, F. J. Tackmann, and H. X. Zhu, *N -jettiness subtractions for $gg \rightarrow H$ at subleading power*, *Phys. Rev. D* **97** (2018) 014013, [[arXiv:1710.03227](#)].
- [29] M. A. Ebert, et al., *Power Corrections for N -Jettiness Subtractions at $\mathcal{O}(\alpha_s)$* , *JHEP* **12** (2018) 084, [[arXiv:1807.10764](#)].
- [30] R. Boughezal, A. Isgro, and F. Petriello, *Next-to-leading-logarithmic power*

- corrections for N -jettiness subtraction in color-singlet production, *Phys. Rev. D* **97** (2018) 076006, [[arXiv:1802.00456](#)].
- [31] R. Boughezal, A. Isgro, and F. Petriello, *Next-to-leading power corrections to $V + 1$ jet production in N -jettiness subtraction*, *Phys. Rev. D* **101** (2020) 016005, [[arXiv:1907.12213](#)].
- [32] M. Czakon, *A novel subtraction scheme for double-real radiation at NNLO*, *Phys. Lett. B* **693** (2010) 259–268, [[arXiv:1005.0274](#)].
- [33] M. Czakon, *Double-real radiation in hadronic top quark pair production as a proof of a certain concept*, *Nucl. Phys. B* **849** (2011) 250–295, [[arXiv:1101.0642](#)].
- [34] M. Czakon and D. Heymes, *Four-dimensional formulation of the sector-improved residue subtraction scheme*, *Nucl. Phys. B* **890** (2014) 152–227, [[arXiv:1408.2500](#)].
- [35] M. Czakon, A. van Hameren, A. Mitov, and R. Poncelet, *Single-jet inclusive rates with exact color at $\mathcal{O}(\alpha_s^4)$* , *JHEP* **10** (2019) 262, [[arXiv:1907.12911](#)].
- [36] F. Caola, K. Melnikov, and R. Röntsch, *Nested soft-collinear subtractions in NNLO QCD computations*, *Eur. Phys. J. C* **77** (2017) 248, [[arXiv:1702.01352](#)].
- [37] F. Caola, K. Melnikov, and R. Röntsch, *Analytic results for color-singlet production at NNLO QCD with the nested soft-collinear subtraction scheme*, *Eur. Phys. J. C* **79** (2019) 386, [[arXiv:1902.02081](#)].
- [38] F. Caola, K. Melnikov, and R. Röntsch, *Analytic results for decays of color singlets to gg and $q\bar{q}$ final states at NNLO QCD with the nested soft-collinear subtraction scheme*, *Eur. Phys. J. C* **79** (2019) 1013, [[arXiv:1907.05398](#)].
- [39] P. Bargiela, F. Caola, H. Chawdhry, and X. Liu, *Two-loop mixed QCD-electroweak amplitudes for Z +jet production at the LHC: bosonic corrections*, *JHEP* **06** (2024) 150, [[arXiv:2312.14145](#)].
- [40] F. Devoto, K. Melnikov, R. Röntsch, C. Signorile-Signorile, and D. M. Tagliabue, *A fresh look at the nested soft-collinear subtraction scheme: NNLO QCD corrections to N -gluon final states in $q\bar{q}$ annihilation*, *JHEP* **02** (2024) 016, [[arXiv:2310.17598](#)].
- [41] L. Magnea, et al., *Local analytic sector subtraction at NNLO*, *JHEP* **12** (2018) 107, [[arXiv:1806.09570](#)]. [Erratum: *JHEP* **06**, 013 (2019)].
- [42] G. Bertolotti, et al., *NNLO subtraction for any massless final state: a complete analytic expression*, *JHEP* **07** (2023) 140, [[arXiv:2212.11190](#)]. [Erratum: *JHEP* **05**, 019 (2024)].
- [43] L. Magnea, G. Pelliccioli, C. Signorile-Signorile, P. Torrielli, and S. Uccirati, *Analytic integration of soft and collinear radiation in factorised QCD cross sections at NNLO*, *JHEP* **02** (2021) 037, [[arXiv:2010.14493](#)].
- [44] A. Gehrmann-De Ridder, T. Gehrmann, and E. W. N. Glover, *Antenna subtraction at NNLO*, *JHEP* **09** (2005) 056, [[hep-ph/0505111](#)].

- [45] A. Gehrmann-De Ridder, T. Gehrmann, and E. W. N. Glover, *Quark-gluon antenna functions from neutralino decay*, *Phys. Lett. B* **612** (2005) 36–48, [[hep-ph/0501291](#)].
- [46] A. Gehrmann-De Ridder, T. Gehrmann, and E. W. N. Glover, *Gluon-gluon antenna functions from Higgs boson decay*, *Phys. Lett. B* **612** (2005) 49–60, [[hep-ph/0502110](#)].
- [47] A. Daleo, T. Gehrmann, and D. Maitre, *Antenna subtraction with hadronic initial states*, *JHEP* **04** (2007) 016, [[hep-ph/0612257](#)].
- [48] A. Daleo, A. Gehrmann-De Ridder, T. Gehrmann, and G. Luisoni, *Antenna subtraction at NNLO with hadronic initial states: initial-final configurations*, *JHEP* **01** (2010) 118, [[arXiv:0912.0374](#)].
- [49] R. Boughezal, A. Gehrmann-De Ridder, and M. Ritzmann, *Antenna subtraction at NNLO with hadronic initial states: double real radiation for initial-initial configurations with two quark flavours*, *JHEP* **02** (2011) 098, [[arXiv:1011.6631](#)].
- [50] T. Gehrmann and P. F. Monni, *Antenna subtraction at NNLO with hadronic initial states: real-virtual initial-initial configurations*, *JHEP* **12** (2011) 049, [[arXiv:1107.4037](#)].
- [51] A. Gehrmann-De Ridder, T. Gehrmann, and M. Ritzmann, *Antenna subtraction at NNLO with hadronic initial states: double real initial-initial configurations*, *JHEP* **10** (2012) 047, [[arXiv:1207.5779](#)].
- [52] J. Currie, E. W. N. Glover, and S. Wells, *Infrared Structure at NNLO Using Antenna Subtraction*, *JHEP* **04** (2013) 066, [[arXiv:1301.4693](#)].
- [53] W. Bernreuther, C. Bogner, and O. Dekkers, *The real radiation antenna function for $S \rightarrow Q\bar{Q}q\bar{q}$ at NNLO QCD*, *JHEP* **06** (2011) 032, [[arXiv:1105.0530](#)].
- [54] W. Bernreuther, C. Bogner, and O. Dekkers, *The real radiation antenna functions for $S \rightarrow Q\bar{Q}gg$ at NNLO QCD*, *JHEP* **10** (2013) 161, [[arXiv:1309.6887](#)].
- [55] G. Abelof and A. Gehrmann-De Ridder, *Antenna subtraction for the production of heavy particles at hadron colliders*, *JHEP* **04** (2011) 063, [[arXiv:1102.2443](#)].
- [56] G. Abelof, O. Dekkers, and A. Gehrmann-De Ridder, *Antenna subtraction with massive fermions at NNLO: Double real initial-final configurations*, *JHEP* **12** (2012) 107, [[arXiv:1210.5059](#)].
- [57] G. Abelof, A. Gehrmann-De Ridder, P. Maierhofer, and S. Pozzorini, *NNLO QCD subtraction for top-antitop production in the $q\bar{q}$ channel*, *JHEP* **08** (2014) 035, [[arXiv:1404.6493](#)].
- [58] J. Currie, et al., *Jet cross sections at the LHC with NNLOJET*, *PoS* **LL2018** (2018) 001, [[arXiv:1807.06057](#)].
- [59] T. Gehrmann, E. W. N. Glover, and M. Marcoli, *The colourful antenna subtraction method*, *JHEP* **03** (2024) 114, [[arXiv:2310.19757](#)].

- [60] O. R. T. Braun-White, *Idealised Antenna Functions for Higher Order QCD Calculations*. PhD thesis, Durham U., 2024. [arXiv:2402.08057](#).
- [61] L. Bonino, T. Gehrmann, M. Marcoli, R. Schürmann, and G. Stagnitto, *Antenna subtraction for processes with identified particles at hadron colliders*, *JHEP* **08** (2024) 073, [[arXiv:2406.09925](#)].
- [62] E. Fox, N. Glover, and M. Marcoli, *Generalised antenna functions for higher-order calculations*, *JHEP* **12** (2025) 225, [[arXiv:2410.12904](#)].
- [63] G. Somogyi, Z. Trocsanyi, and V. Del Duca, *Matching of singly- and doubly-unresolved limits of tree-level QCD squared matrix elements*, *JHEP* **06** (2005) 024, [[hep-ph/0502226](#)].
- [64] G. Somogyi, Z. Trocsanyi, and V. Del Duca, *A Subtraction scheme for computing QCD jet cross sections at NNLO: Regularization of doubly-real emissions*, *JHEP* **01** (2007) 070, [[hep-ph/0609042](#)].
- [65] G. Somogyi and Z. Trocsanyi, *A Subtraction scheme for computing QCD jet cross sections at NNLO: Regularization of real-virtual emission*, *JHEP* **01** (2007) 052, [[hep-ph/0609043](#)].
- [66] U. Aglietti, V. Del Duca, C. Duhr, G. Somogyi, and Z. Trocsanyi, *Analytic integration of real-virtual counterterms in NNLO jet cross sections. I.*, *JHEP* **09** (2008) 107, [[arXiv:0807.0514](#)].
- [67] V. Del Duca, G. Somogyi, and Z. Trocsanyi, *Integration of collinear-type doubly unresolved counterterms in NNLO jet cross sections*, *JHEP* **06** (2013) 079, [[arXiv:1301.3504](#)].
- [68] M. Cacciari, F. A. Dreyer, A. Karlberg, G. P. Salam, and G. Zanderighi, *Fully Differential Vector-Boson-Fusion Higgs Production at Next-to-Next-to-Leading Order*, *Phys. Rev. Lett.* **115** (2015) 082002, [[arXiv:1506.02660](#)]. [Erratum: *Phys.Rev.Lett.* 120, 139901 (2018)].
- [69] J. Currie, et al., *N^3LO corrections to jet production in deep inelastic scattering using the Projection-to-Born method*, *JHEP* **05** (2018) 209, [[arXiv:1803.09973](#)].
- [70] G. P. Lepage, *A New Algorithm for Adaptive Multidimensional Integration*, *J. Comput. Phys.* **27** (1978) 192.
- [71] S. Badger et al., *Machine learning and LHC event generation*, *SciPost Phys.* **14** (2023) 079, [[arXiv:2203.07460](#)].
- [72] E. Bothmann, T. Janßen, M. Knobbe, T. Schmale, and S. Schumann, *Exploring phase space with Neural Importance Sampling*, *SciPost Phys.* **8** (2020) 069, [[arXiv:2001.05478](#)].
- [73] C. Gao, S. Höche, J. Isaacson, C. Krause, and H. Schulz, *Event Generation with Normalizing Flows*, *Phys. Rev. D* **101** (2020) 076002, [[arXiv:2001.10028](#)].
- [74] E. G. Tabak and E. Vanden-Eijnden, *Density estimation by dual ascent of the log-likelihood*, *Communications in Mathematical Sciences* **8** (2010) 217 – 233.

- [75] E. G. Tabak and C. V. Turner, *A Family of Nonparametric Density Estimation Algorithms*, *Commun. Pure Appl. Math.* **66** (2013) 145–164.
- [76] D. Rezende and S. Mohamed, *Variational inference with normalizing flows*, in *Proceedings of the 32nd International Conference on Machine Learning* (F. Bach and D. Blei, eds.), vol. 37 of *Proceedings of Machine Learning Research*, (Lille, France), pp. 1530–1538, PMLR, 07–09 Jul, 2015.
- [77] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, *Normalizing flows for probabilistic modeling and inference*, *Journal of Machine Learning Research* **22** (2021) 1–64.
- [78] **Sherpa** Collaboration, E. Bothmann et al., *Event generation with Sherpa 3*, *JHEP* **12** (2024) 156, [[arXiv:2410.22148](#)].
- [79] **Sherpa** Collaboration, E. Bothmann et al., *Event Generation with Sherpa 2.2*, *SciPost Phys.* **7** (2019) 034, [[arXiv:1905.09127](#)].
- [80] T. Heimgel, et al., *MadNIS - Neural multi-channel importance sampling*, *SciPost Phys.* **15** (2023) 141, [[arXiv:2212.06172](#)].
- [81] T. Heimgel, et al., *The MadNIS Reloaded*, *SciPost Phys.* **17** (2024) 023, [[arXiv:2311.01548](#)].
- [82] T. Heimgel, O. Mattelaer, T. Plehn, and R. Winterhalder, *Differentiable MadNIS-Lite*, *SciPost Phys.* **18** (2025) 017, [[arXiv:2408.01486](#)].
- [83] J. Alwall, M. Herquet, F. Maltoni, O. Mattelaer, and T. Stelzer, *MadGraph 5 : Going Beyond*, *JHEP* **06** (2011) 128, [[arXiv:1106.0522](#)].
- [84] J. Alwall, et al., *The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations*, *JHEP* **07** (2014) 079, [[arXiv:1405.0301](#)].
- [85] C. Gao, J. Isaacson, and C. Krause, *i-flow: High-dimensional Integration and Sampling with Normalizing Flows*, *Mach. Learn. Sci. Tech.* **1** (2020) 045023, [[arXiv:2001.05486](#)].
- [86] R. Verheyen, *Event Generation and Density Estimation with Surjective Normalizing Flows*, *SciPost Phys.* **13** (2022) 047, [[arXiv:2205.01697](#)].
- [87] N. Deutschmann and N. Götz, *Accelerating HEP simulations with Neural Importance Sampling*, *JHEP* **03** (2024) 083, [[arXiv:2401.09069](#)].
- [88] A. Kofler, V. Stimper, M. Mikhasenko, M. Kagan, and L. Heinrich, *Flow Annealed Importance Sampling Bootstrap meets Differentiable Particle Physics*, in *38th conference on Neural Information Processing Systems*, 11, 2024. [[arXiv:2411.16234](#)].
- [89] M. El Baz, et al., *Efficient Monte Carlo event generation for neutrino-nucleus exclusive cross sections*, *Phys. Rev. D* **111** (2025) 113001, [[arXiv:2502.14452](#)].

- [90] M. Czakon, Z. Kassabov, A. Mitov, R. Poncelet, and A. Popescu, *HighTEA: high energy theory event analyser*, *J. Phys. G* **51** (2024) 115002, [[arXiv:2304.05993](#)].
- [91] K. Danziger, T. Janßen, S. Schumann, and F. Siegert, *Accelerating Monte Carlo event generation – rejection sampling using neural network event-weight estimates*, *SciPost Phys.* **12** (2022) 164, [[arXiv:2109.11964](#)].
- [92] T. Janßen, D. Maître, S. Schumann, F. Siegert, and H. Truong, *Unweighting multijet event generation using factorisation-aware neural networks*, *SciPost Phys.* **15** (2023) 107, [[arXiv:2301.13562](#)].
- [93] R. Winterhalder, et al., *Targeting multi-loop integrals with neural networks*, *SciPost Phys.* **12** (2022) 129, [[arXiv:2112.09145](#)].
- [94] R. Jinno, G. Kälin, Z. Liu, and H. Rubira, *Machine learning Post-Minkowskian integrals*, *JHEP* **07** (2023) 181, [[arXiv:2209.01091](#)].
- [95] M. Czakon, D. Heymes, and A. Mitov, *High-precision differential predictions for top-quark pairs at the LHC*, *Phys. Rev. Lett.* **116** (2016) 082003, [[arXiv:1511.00549](#)].
- [96] L. Dinh, D. Krueger, and Y. Bengio, *NICE: non-linear independent components estimation*, in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Workshop Track Proceedings* (Y. Bengio and Y. LeCun, eds.), 2015. [arXiv:1410.8516](#).
- [97] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, *Neural ordinary differential equations*, in *Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS’18*, (Red Hook, NY, USA), p. 6572–6583, Curran Associates Inc., 2018.
- [98] E. Bothmann, T. Janßen, M. Knobbe, B. Schmitzer, and F. Sinz, “Efficient many-jet event generation with flow matching.” manuscript in preparation, 2025.
- [99] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, *Flow matching for generative modeling*, in *The Eleventh International Conference on Learning Representations*, 2023. [arXiv:2210.02747](#).
- [100] A. Tong, et al., *Improving and generalizing flow-based generative models with minibatch optimal transport*, *Transactions on Machine Learning Research* (2024). Expert Certification.
- [101] E. Bothmann, et al., *A portable parton-level event generator for the high-luminosity LHC*, *SciPost Phys.* **17** (2024) 081, [[arXiv:2311.06198](#)].
- [102] A. van Hameren, *PARNI for importance sampling and density estimation*, *Acta Phys. Polon. B* **40** (2009) 259–272, [[arXiv:0710.2448](#)].
- [103] D. Kroese, T. Taimre, and Z. Botev, *Handbook of Monte Carlo Methods*. Wiley Series in Probability and Statistics. Wiley, 2013.
- [104] L. Devroye, *Non-Uniform Random Variate Generation*. Springer, New York, 1986.

- [105] J. von Neumann, *Various techniques used in connection with random digits*, in *Monte Carlo Method* (A. S. Householder, G. E. Forsythe, and H. H. Germond, eds.), vol. 12 of *National Bureau of Standards Applied Mathematics Series*, ch. 13, pp. 36–38. US Government Printing Office, Washington, DC, 1951.
- [106] R. Frederix, S. Frixione, S. Prestel, and P. Torrielli, *On the reduction of negative weights in MC@NLO-type matching procedures*, *JHEP* **07** (2020) 238, [[arXiv:2002.12716](#)].
- [107] J. R. Andersen, C. Gütschow, A. Maier, and S. Prestel, *A Positive Resampler for Monte Carlo events with negative weights*, *Eur. Phys. J. C* **80** (2020) 1007, [[arXiv:2005.09375](#)].
- [108] K. Danziger, S. Höche, and F. Siegert, *Reducing negative weights in Monte Carlo event generation with Sherpa*, [arXiv:2110.15211](#).
- [109] R. Frederix and P. Torrielli, *A new way of reducing negative weights in MC@NLO*, *Eur. Phys. J. C* **83** (2023) 1051, [[arXiv:2310.04160](#)].
- [110] T. Müller, B. McWilliams, F. Rousselle, M. Gross, and J. Novák, *Neural importance sampling*, *ACM Trans. Graph.* **38** (2019) [[arXiv:1808.03856](#)].
- [111] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios, *Neural spline flows*, in *Advances in Neural Information Processing Systems* (H. Wallach, et al., eds.), vol. 32, Curran Associates, Inc., 2019. [arXiv:1906.04032](#).
- [112] S. Kullback and R. A. Leibler, *On Information and Sufficiency*, *The Annals of Mathematical Statistics* **22** (1951) 79–86.
- [113] T. Akhond-Sadeh, et al., *Iterated denoising energy matching for sampling from boltzmann densities*, in *Proceedings of the 41st International Conference on Machine Learning*, ICML’24, JMLR.org, 2024. [arXiv:2402.06121](#).
- [114] M. S. Albergo and E. Vanden-Eijnden, *Building normalizing flows with stochastic interpolants*, in *The Eleventh International Conference on Learning Representations*, 2023. [arXiv:2209.15571](#).
- [115] X. Liu, C. Gong, and Q. Liu, *Flow straight and fast: Learning to generate and transfer data with rectified flow*, in *The Eleventh International Conference on Learning Representations*, 2023. [arXiv:2209.03003](#).
- [116] NNPDF Collaboration, R. D. Ball et al., *Parton distributions from high-precision collider data*, *Eur. Phys. J. C* **77** (2017) 663, [[arXiv:1706.00428](#)].
- [117] G. P. Lepage, “vegas.” <https://github.com/gplepage/vegas>.
- [118] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios, *nflows: normalizing flows in PyTorch*. Zenodo, 2020. <https://doi.org/10.5281/zenodo.4296287>.
- [119] A. Tong, et al., *Simulation-free schrödinger bridges via score and flow matching*, [arXiv:2307.03672](#).
- [120] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, *Self-normalizing neural*

- networks, in *Advances in Neural Information Processing Systems* (I. Guyon, et al., eds.), vol. 30, Curran Associates, Inc., 2017.
- [121] J. Wildberger, et al., *Flow matching for scalable simulation-based inference*, in *Advances in Neural Information Processing Systems* (A. Oh, et al., eds.), vol. 36, pp. 16837–16864, Curran Associates, Inc., 2023.
- [122] M. Poli, et al., “Torchdyn: Implicit models and neural numerical methods in pytorch.” <https://github.com/DiffEqML/torchdyn>.
- [123] J. Dormand and P. Prince, *A family of embedded runge-kutta formulae*, *Journal of Computational and Applied Mathematics* **6** (1980) 19–26.
- [124] A. Paszke, et al., *Pytorch: An imperative style, high-performance deep learning library*, in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035. Curran Associates, Inc., 2019.
- [125] I. Loshchilov and F. Hutter, *Decoupled weight decay regularization*, in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, OpenReview.net, 2019.